



Reinforcement Learning

Autumn 2024

Abhishek Gupta

TA: Jacob Berg



Lecture outline

Recap: MDP formalism + why should we care?



Imitation learning: preliminaries and behavior cloning



Multimodality and Underfitting in Imitation



Compounding Error in Imitation

Ok so let's try and define Reinforcement Learning

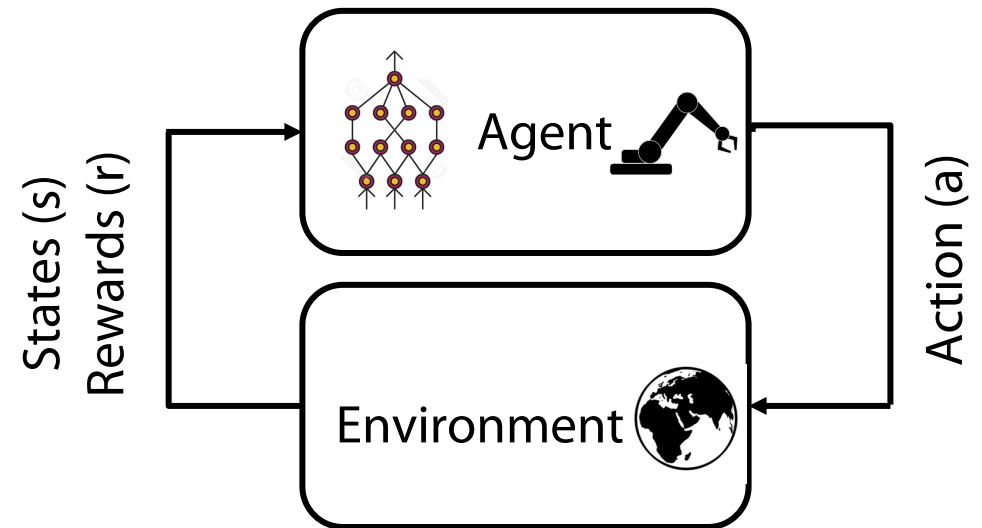
Can learn in arbitrary settings

Go from expert label $\rightarrow$
scalar measure of success

No expert corpus needed

Can generalize to new states

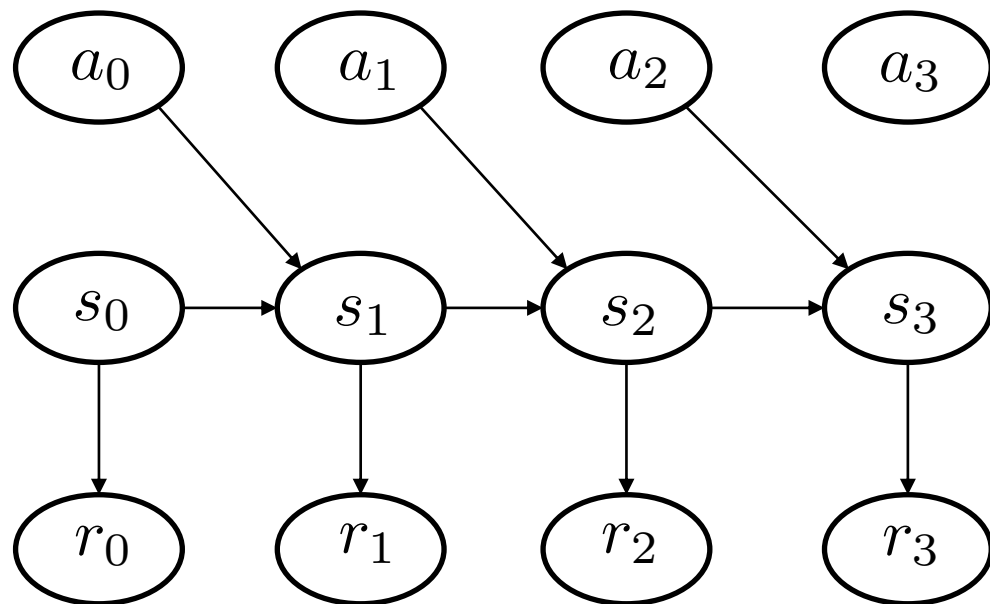
Using **trial and error** in an **environment** to learn a strategy to maximize some notion of **"reward"**



Easy (?) way for agents to continue improving their own behavior on deployment

Framework for RL - Markov Decision Process

Augment Markov chain with rewards and actions



States: $\mathcal{S}$

Initial state dist: $\rho_0(s)$

Actions: $\mathcal{A}$

Discount: γ

Rewards: $\mathcal{R}$

Transition Dynamics - $p(s_{t+1}|s_t, a_t)$

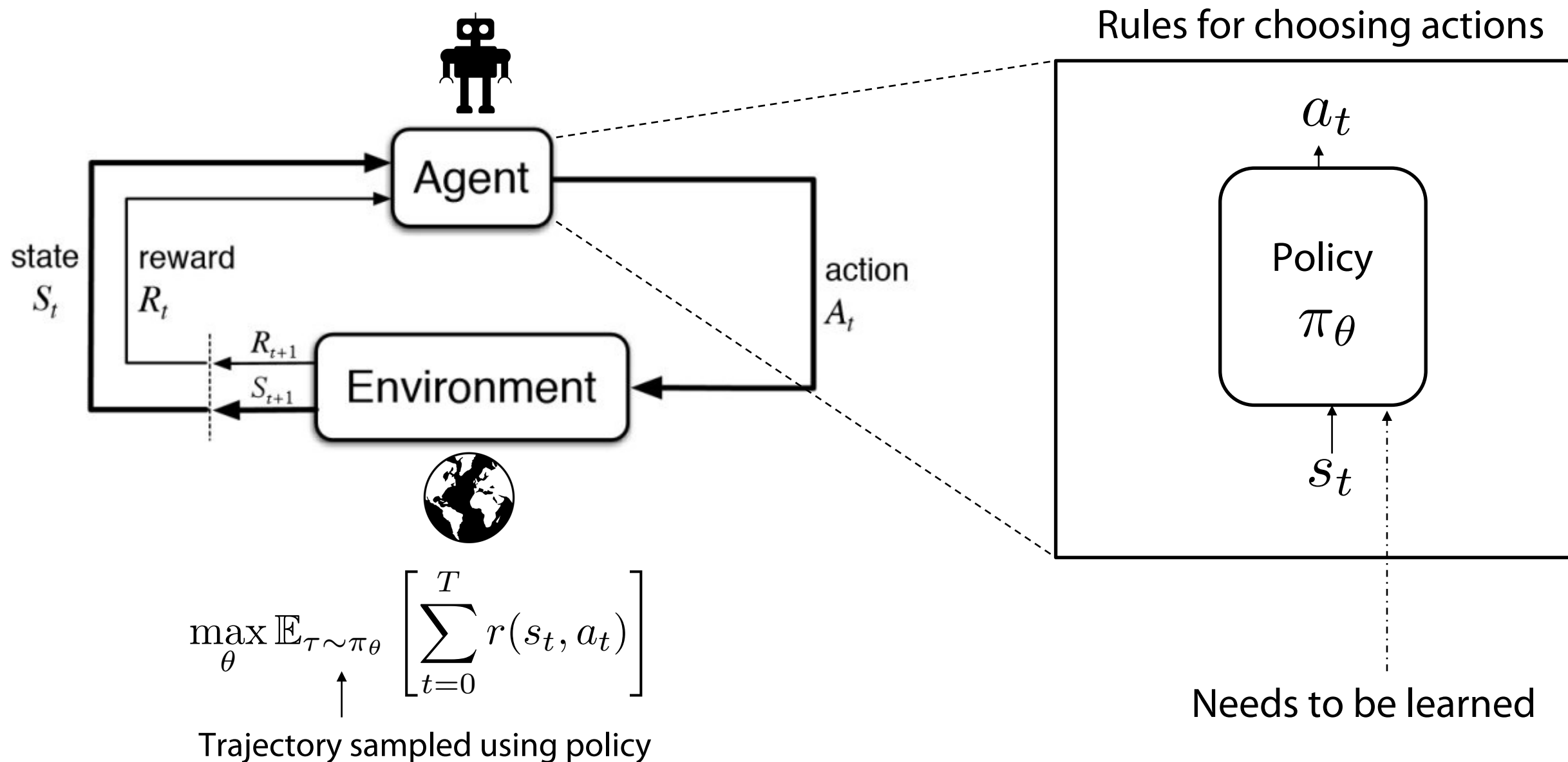
Markov property

$$p(s_1, s_2, s_3) = p(s_3|s_2)p(s_2|s_1)p(s_1)$$

Trajectory

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$$

Reinforcement Learning Formalism



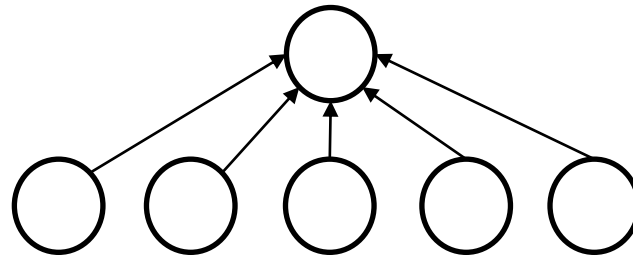
Main thing to learn - Policies

Policies are **mappings** from states to distributions over actions

Tabular

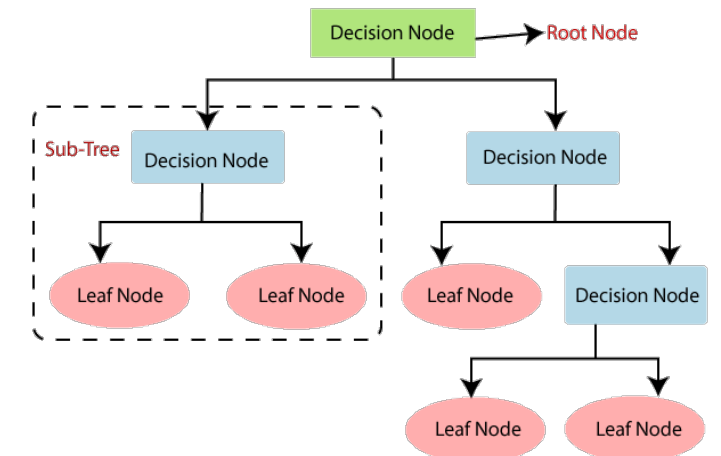
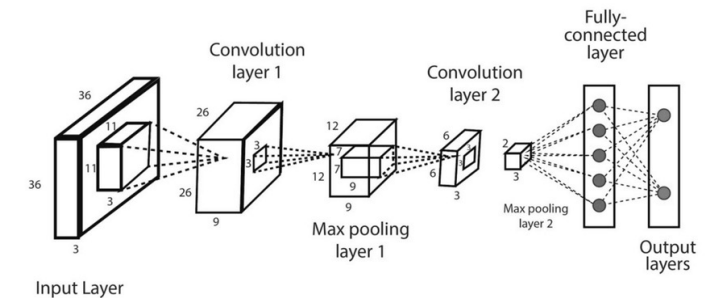
8.67	8.93	9.11	9.30	9.42
8.49		9.09	9.42	9.68
8.33		1.00		10.00
7.13	5.04	3.15	5.68	8.45
-10.00	-10.00	-10.00	-10.00	-10.00

Linear



$$\pi(a|s) = \langle \phi(s, a), w \rangle$$

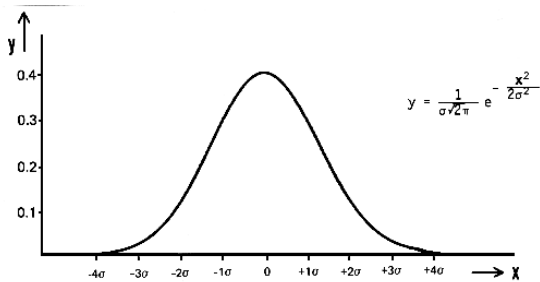
Arbitrary function approx



Main thing to learn - Policies

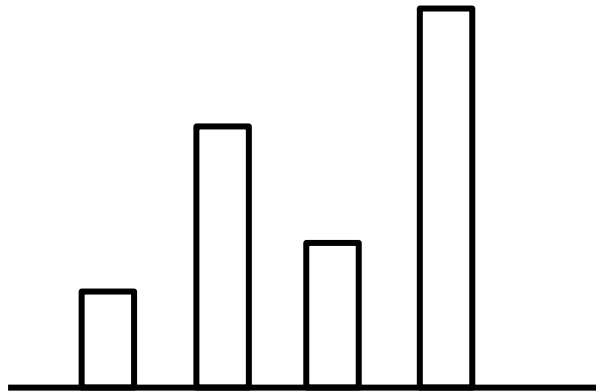
Policies are mappings from states to **distributions** over actions

Gaussian



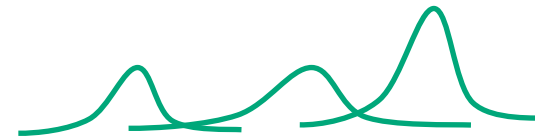
$$\mu(s), \Sigma(s)$$

Categorical



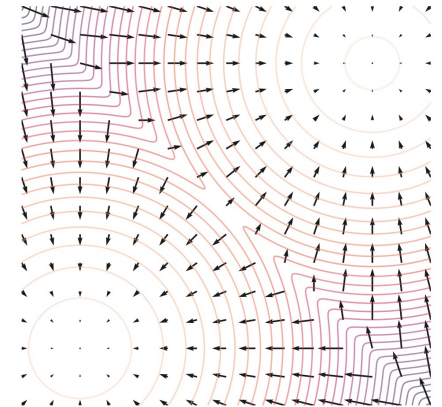
$$p_1(s), p_2(s), \dots, p_k(s)$$

Mixture of Gaussians



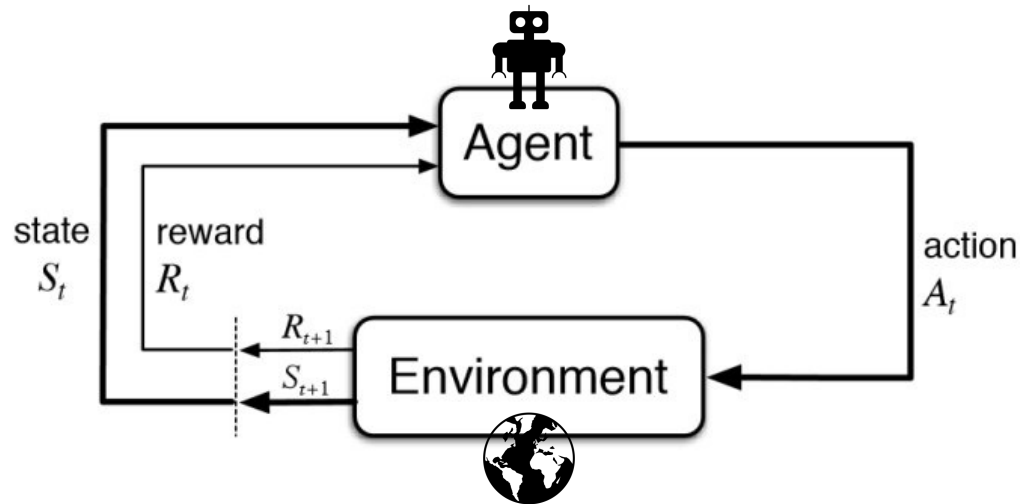
$$\begin{aligned} &\mu_1(s), \Sigma_1(s), w_1 \\ &\mu_2(s), \Sigma_2(s), w_2 \\ &\dots \\ &\mu_N(s), \Sigma_N(s), w_N \end{aligned}$$

Diffusion Models



$$\nabla_a \log \pi(a|s)$$

Let's revisit policies: stochastic vs deterministic



$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$

Lemma 1: Every MDP has atleast one optimal *deterministic* policy

Theorem 1.7. Let Π be the set of all non-stationary and randomized policies. Define:

$$V^*(s) := \sup_{\pi \in \Pi} V^{\pi}(s)$$
$$Q^*(s, a) := \sup_{\pi \in \Pi} Q^{\pi}(s, a).$$

which is finite since $V^{\pi}(s)$ and $Q^{\pi}(s, a)$ are bounded between 0 and $1/(1 - \gamma)$.

There exists a stationary and deterministic policy π such that for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$,

$$V^{\pi}(s) = V^*(s)$$
$$Q^{\pi}(s, a) = Q^*(s, a).$$

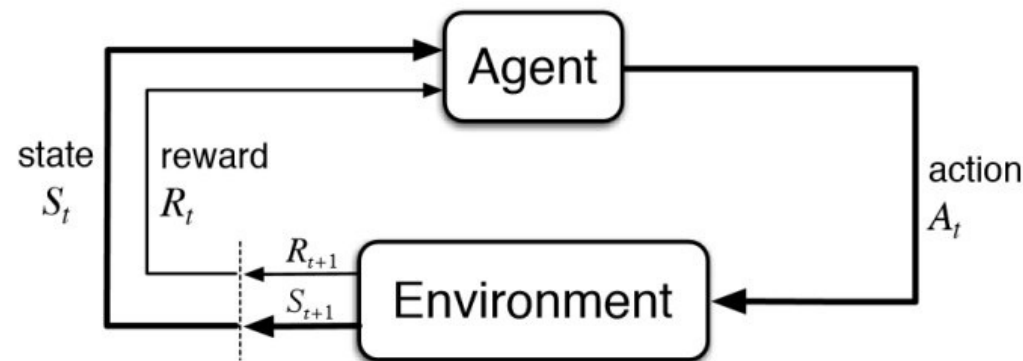
We refer to such a π as an optimal policy.

Intuition: pick the best possible action at every state

Stochastic policies will help in the search/optimization process for finding (close to) deterministic policies

Let's take a closer look at the objective: horizon ⁷

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$



Finite horizon

$$\mathbb{E}_{\pi_{\theta}^t} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$

Time-dependent policy
(not stationary)

$$\mathbb{E}_{\pi_{\theta}^t} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right]$$

Infinite horizon discounted

$$\mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$$

Time-independent (stationary) policy
→ Need discount to prevent blow up

Lemma: there always exists a stationary optimal policy

Unpacking the Expectation

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$

Trajectory View - Ancestral sampling along MDP

Initial state	$\mathbb{E}_{s_0 \sim \rho_0(s)}$	$\left[\sum_{t=0}^T r(s_t, a_t) \right]$
Policy	$a_0 \sim \pi_{\theta}(\cdot s_0)$	
Dynamics	$s_1 \sim p(\cdot s_0, a_0)$	
Policy	$a_1 \sim \pi_{\theta}(\cdot s_1)$	
Dynamics	$s_2 \sim p(\cdot s_1, a_1)$	
	...	

Compact representation	$\mathbb{E}_{\substack{s_0 \sim \rho_0(s) \\ a_t \sim \pi_{\theta}(\cdot s_t) \\ s_{t+1} \sim p(s_{t+1} s_t, a_t)}} \left[\sum_{t=0}^T r(s_t, a_t) \right]$
------------------------	--

$$\mathbb{E}_{\pi_{\theta}^t} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$$

Stationary View – sampling from stationary dist

$$d_t^{\pi}(s, a) = \mathbb{P}(s_t = s, a_t = a \mid s_0 \sim \rho_0, \forall i < t, a_i \sim \pi_{\theta}(\cdot | s_i), s_{i+1} \sim p(\cdot | s_i, a_i))$$

(Likelihood of being at state s, action a at time step t)

$$\mu_{\pi}^{\gamma}(s, a) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t d_t^{\pi}(s, a)$$

(Likelihood of being at state s, action a across **all** steps)

γ subsumed into E	$\mathbb{E}_{(s,a) \sim \mu_{\gamma}^{\pi}(s,a)} \left[r(s, a) \right]$
--------------------------	--

No sequential sampling

No sum over rewards

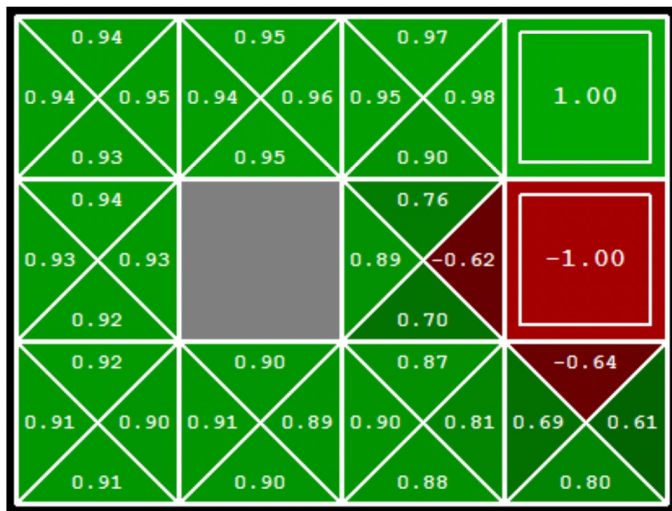
Some notation: Q-functions and V-functions

Estimate of how “good” a policy is – estimate of future returns under a policy π

Q-function

Take one action and then follow policy from s

$$Q^\pi(s, a) = \mathbb{E}_{\pi, p} \left[\sum_t r(s_t, a_t) \mid s_0 = s, a_0 = a \right]$$



V-function

Follow policy from s

$$V^\pi(s, a) = \mathbb{E}_{\pi, p} \left[\sum_t r(s_t, a_t) \mid s_0 = s \right]$$

$$V^\pi(s, a) = \mathbb{E}_{a \sim \pi(\cdot | s)} [Q^\pi(s, a)]$$

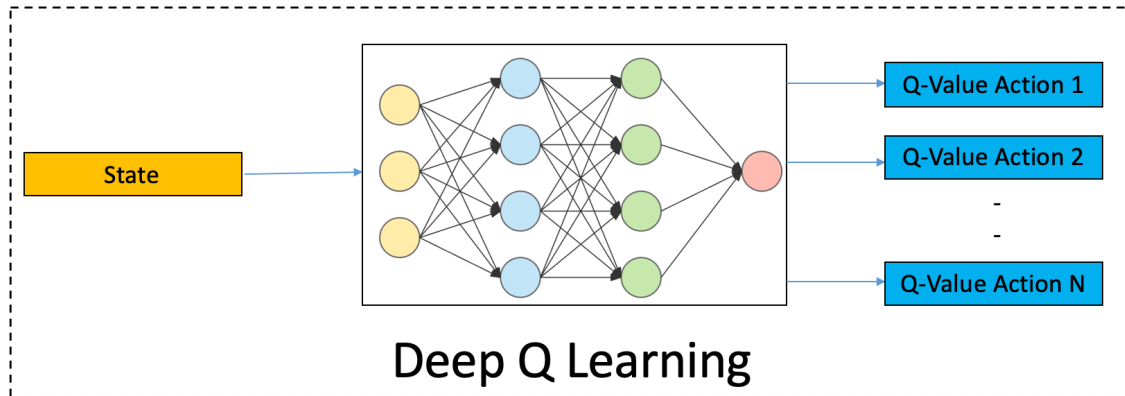
$$J(\pi) = \mathbb{E}_{s \sim \rho_0(s)} [V^\pi(s)]$$

Average value over initial states

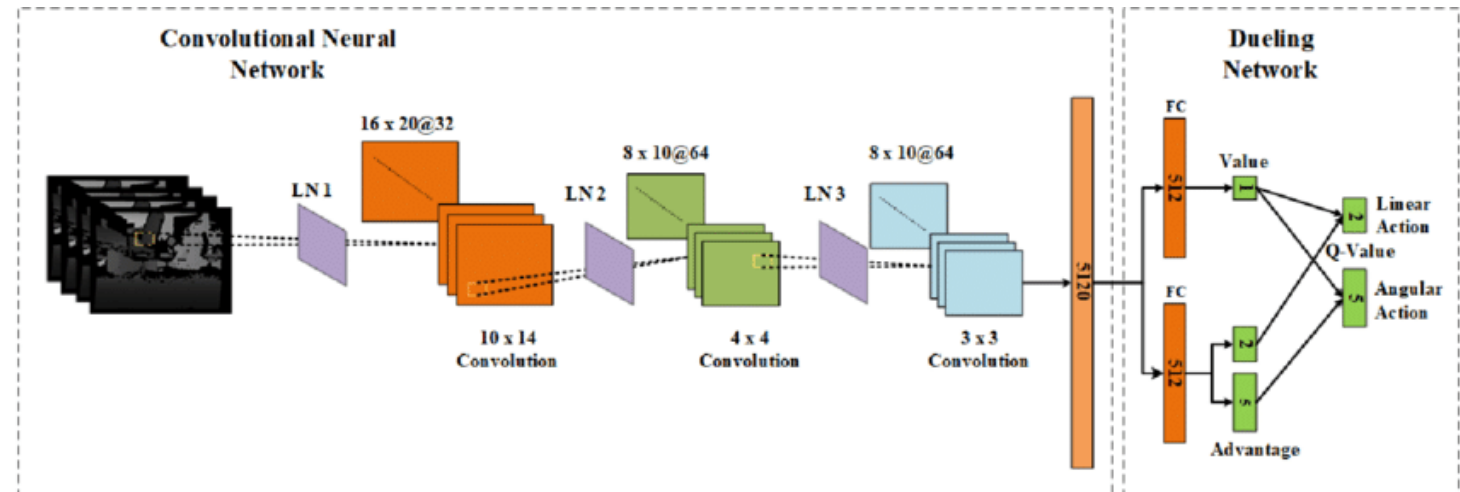
Will be useful soon!

Ok so where does deep learning fit in?

Avoids expensive hand-design for adaptive agents, learn end-to-end: sensors $\rightarrow$ actions



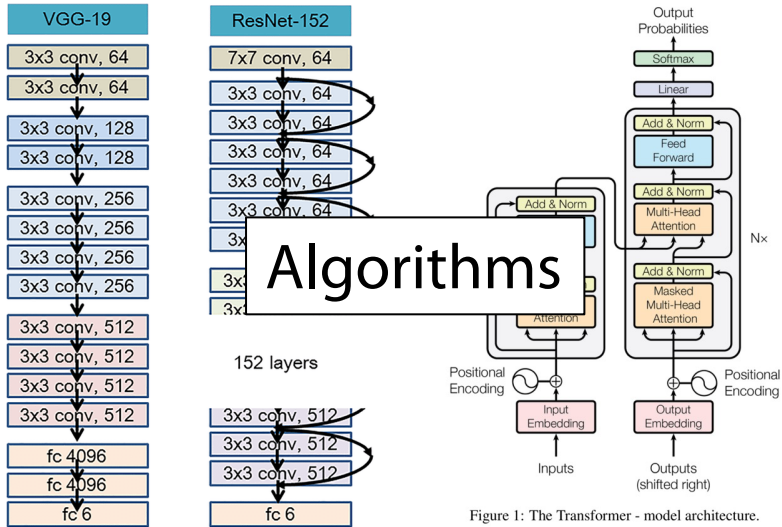
Policies/Q-values/model are represented as deep neural networks



Leads to non-trivial challenges in learning and optimization!

Why is deep RL important now?

Deep models have enabled the huge advances in modern AI

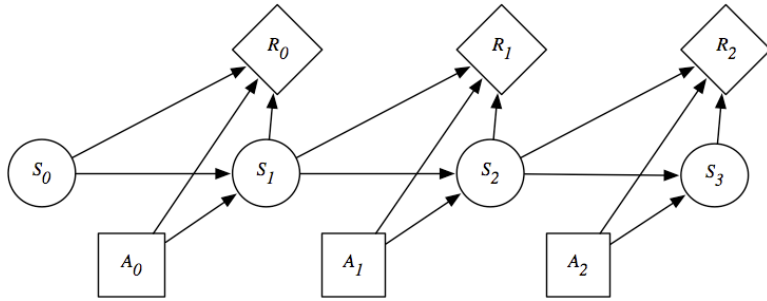


We are betting that the same holds for RL

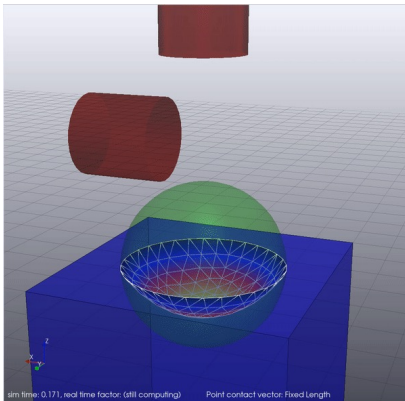


Ok so why should we care about RL?

Solves sequential decision making problems



Has black-box assumptions



Enables continual improvement



Reduces burden of human data collection

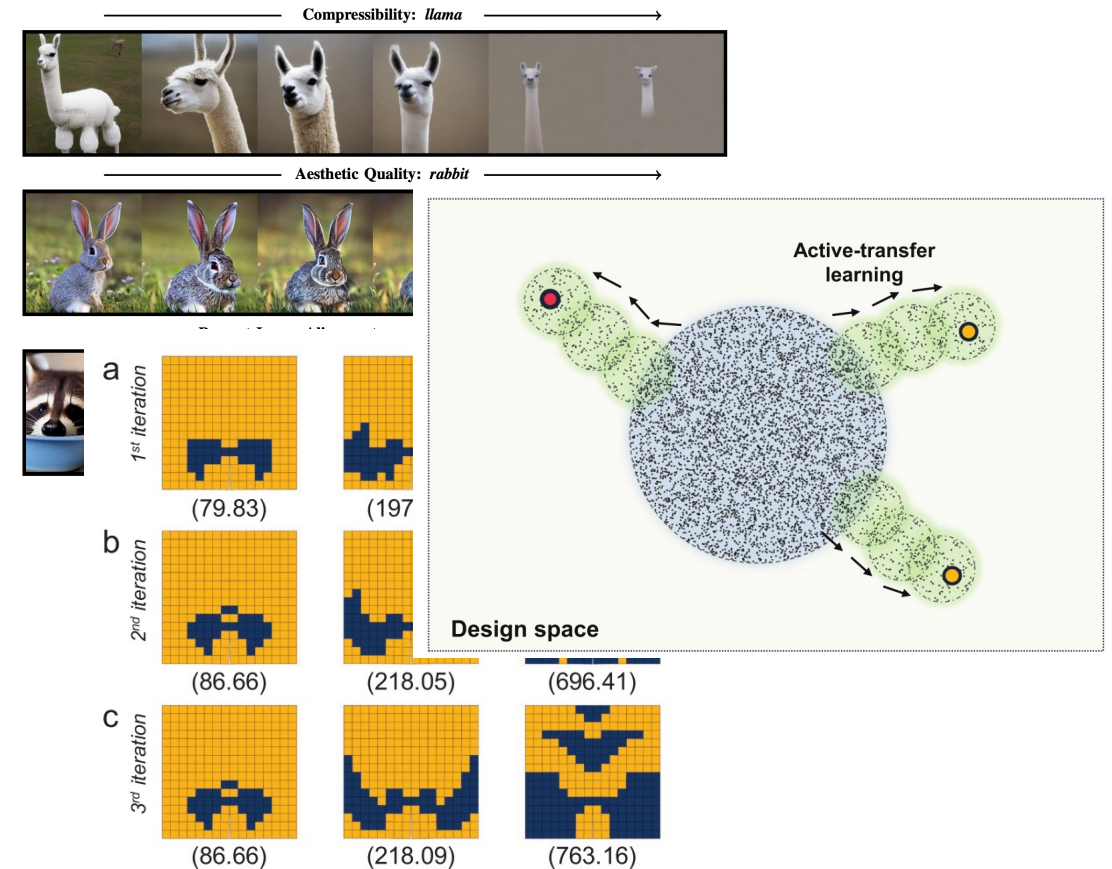


Reinforcement Learning enables going beyond the data

Generative AI is inherently about **replicating** the data distribution

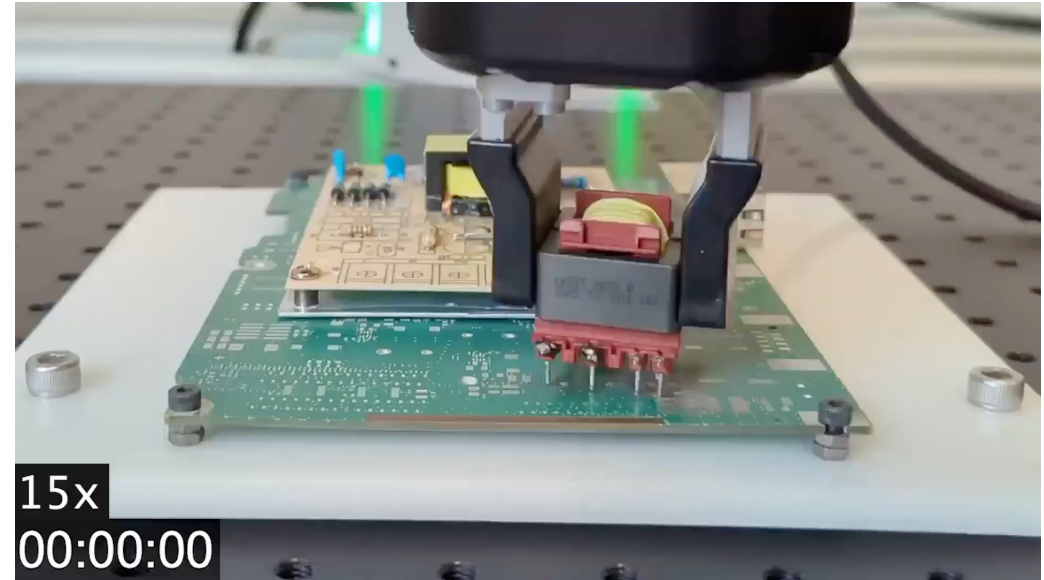
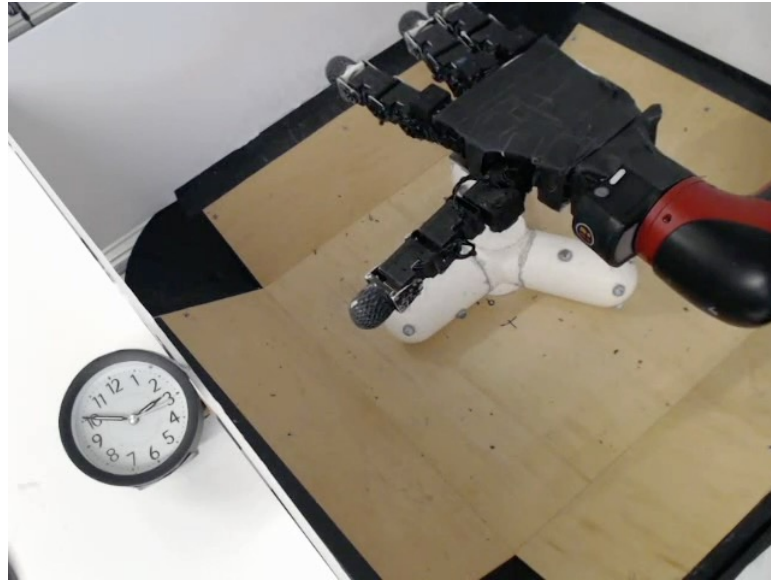


Reinforcement learning can go beyond the data



Applications of RL: Robotics

RL can enable robotic learning of hard to specify/script behaviors in the presence of contact



Applications of RL: Large Language Models

Systematically finds and reduces model hallucinations using RLHF

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A Explain gravity... B Explain war... C Moon is natural satellite of... D People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

Write a story about frogs

The policy generates an output.

PPO

Once upon a time...

The reward model calculates a reward for the output.

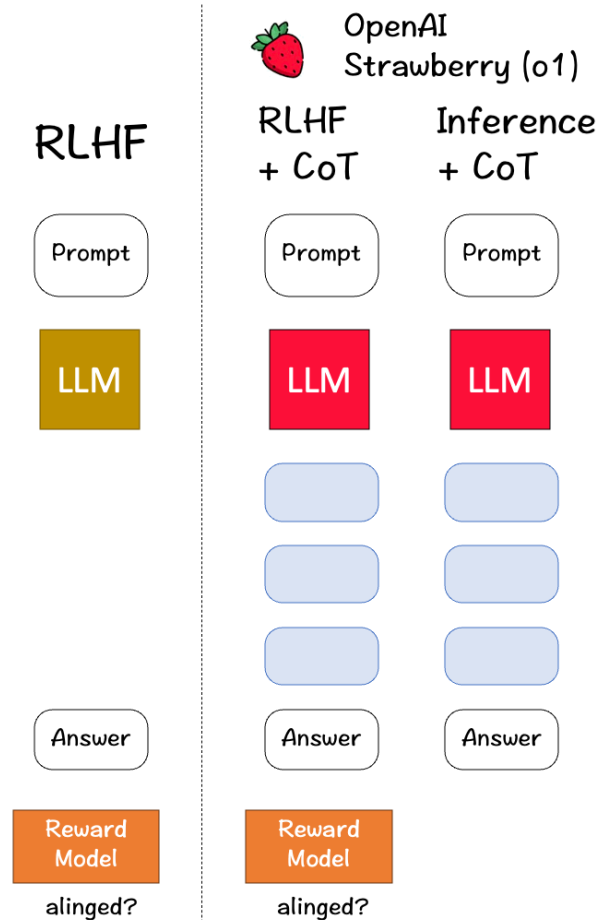
RM

The reward is used to update the policy using PPO.

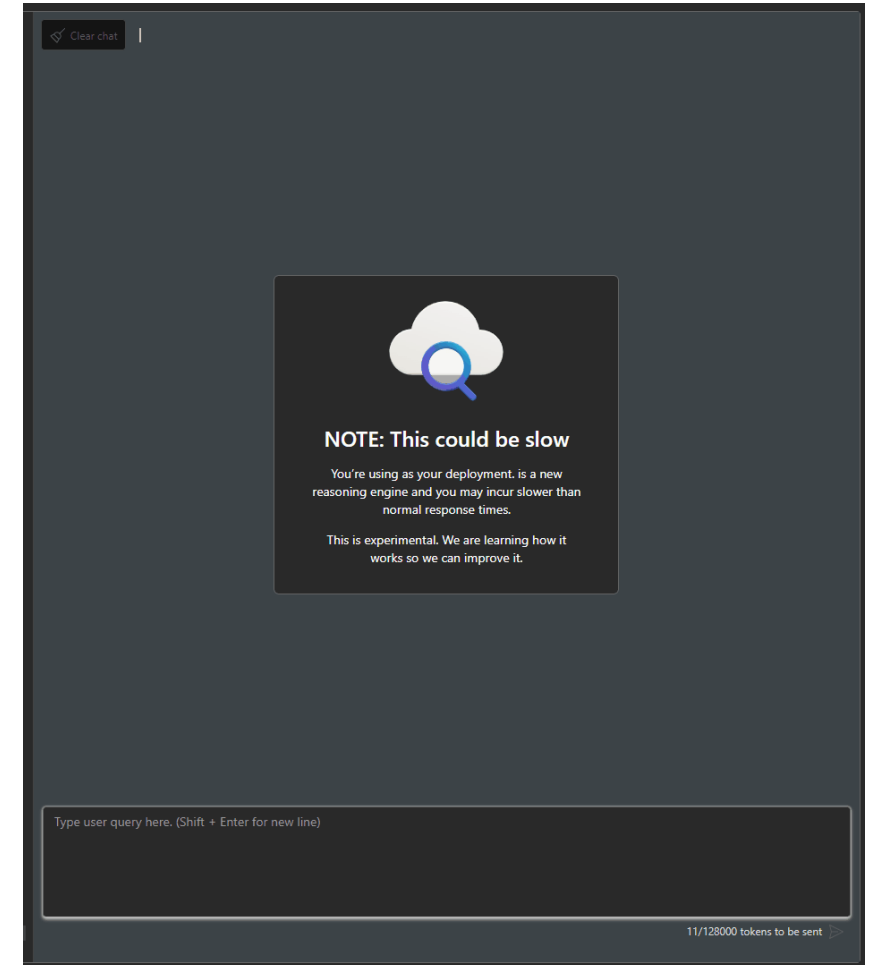
r_k

Applications of RL: Large Language Models

New reasoning-style LLM algorithms are typically RL based

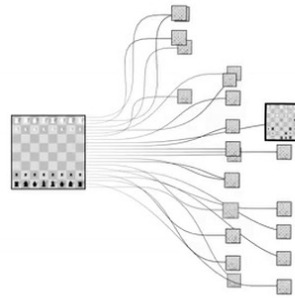


Looks a lot like model-based RL



Applications of RL: Games

Both single and multi-agent RL has proven transformative for game AI

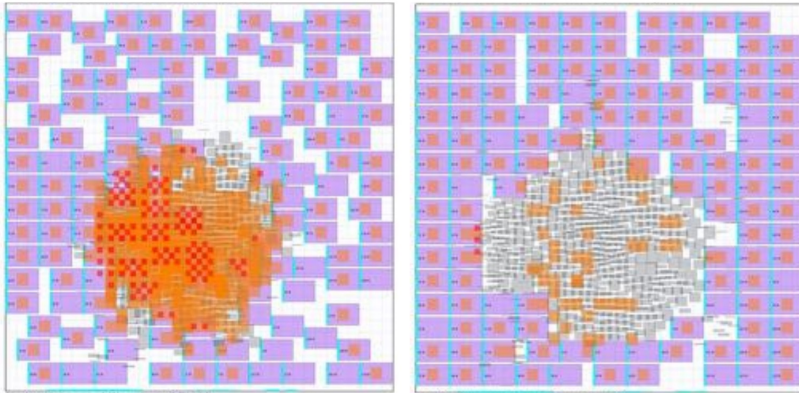


Particularly well suited to RL assumptions

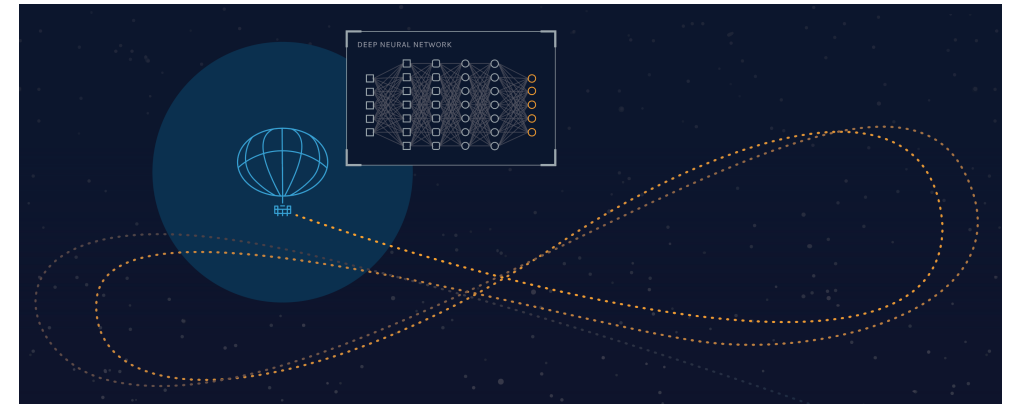
Applications of RL: Science and Engineering

RL has started to become a useful tool for engineering design

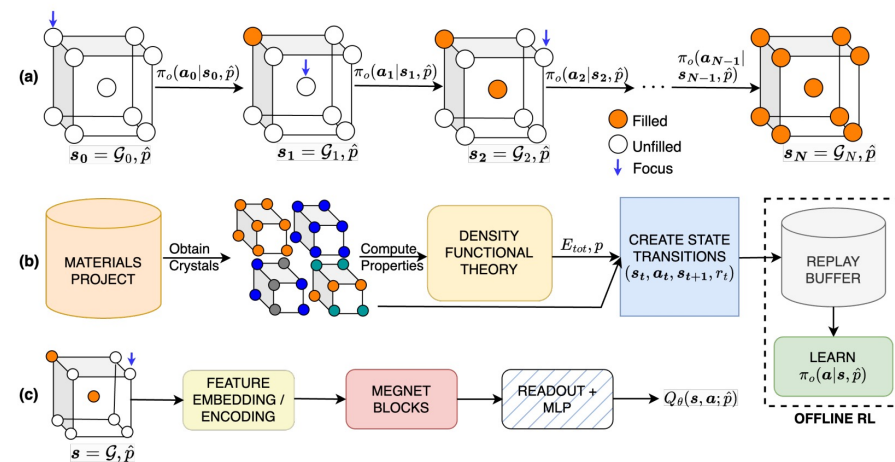
Chip Design



Weather balloon navigation

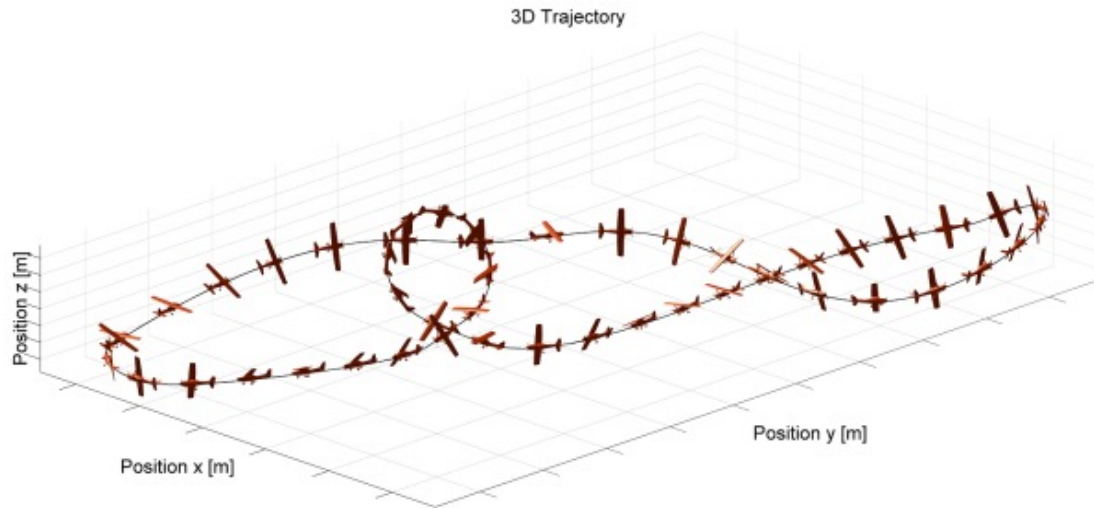


Crystal design



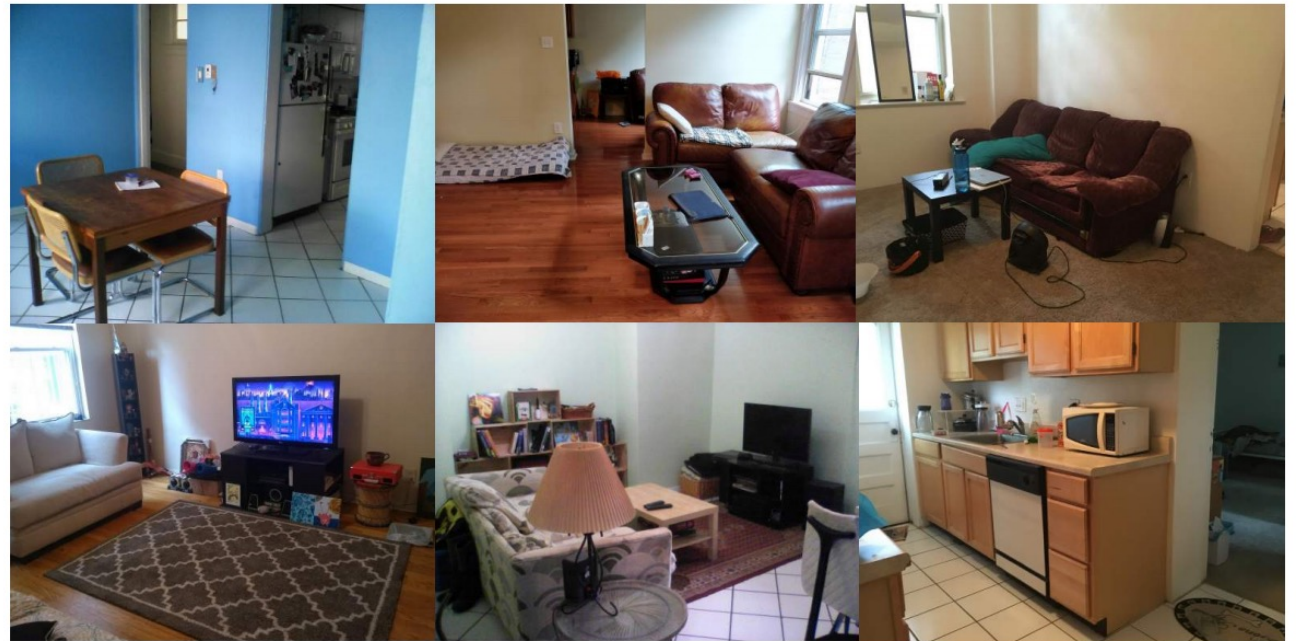
Where is Reinforcement Learning not useful?

Not the right call for very safety-critical, repetitive applications



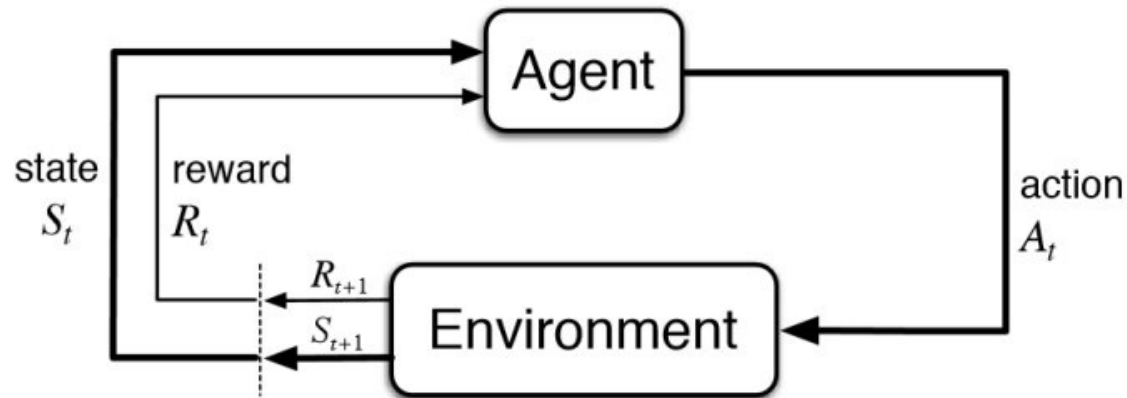
Where is Reinforcement Learning “potentially” useful?

Domains which have high diversity, yet relatively cheap autonomous data collection



But these domains are not as simple as just running RL algorithms!

So is sequential decision making = RL?

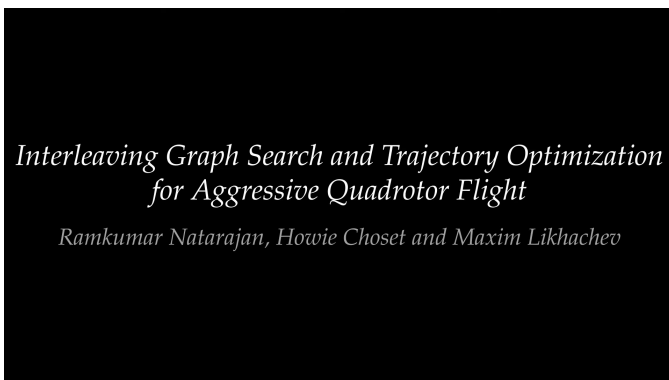


We conflated sequential decision making and RL!

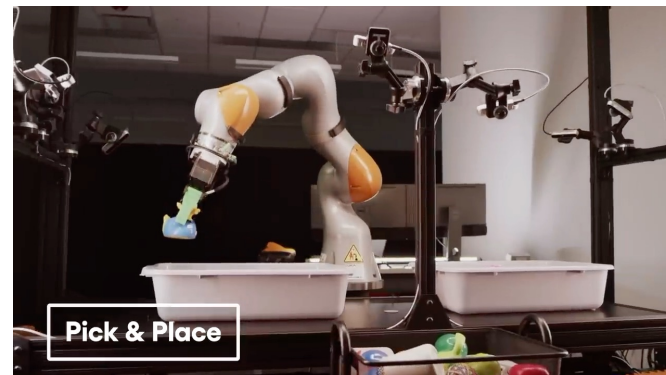
RL is sequential decision making under a particular set of assumptions:

1. Sampling access to the environment
2. Access to reward
3. Goal-directed behavior

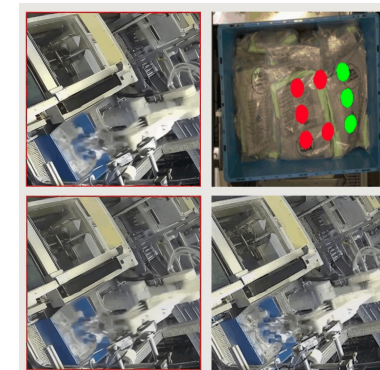
Trajectory optimization/planning



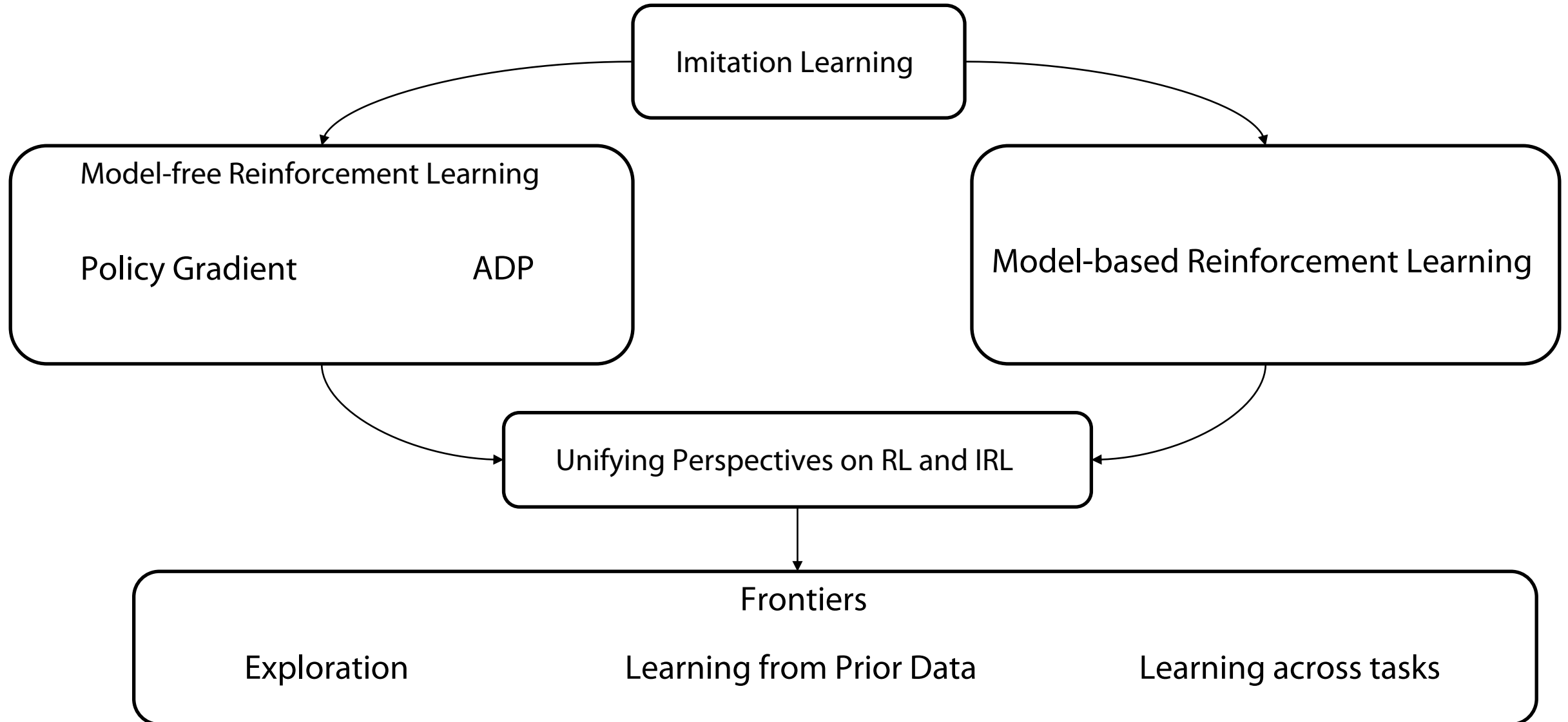
Imitation Learning



Unsupervised Decision Making



Class Structure



Lecture outline

Recap: MDP formalism + why should we care?



Imitation learning: preliminaries and behavior cloning

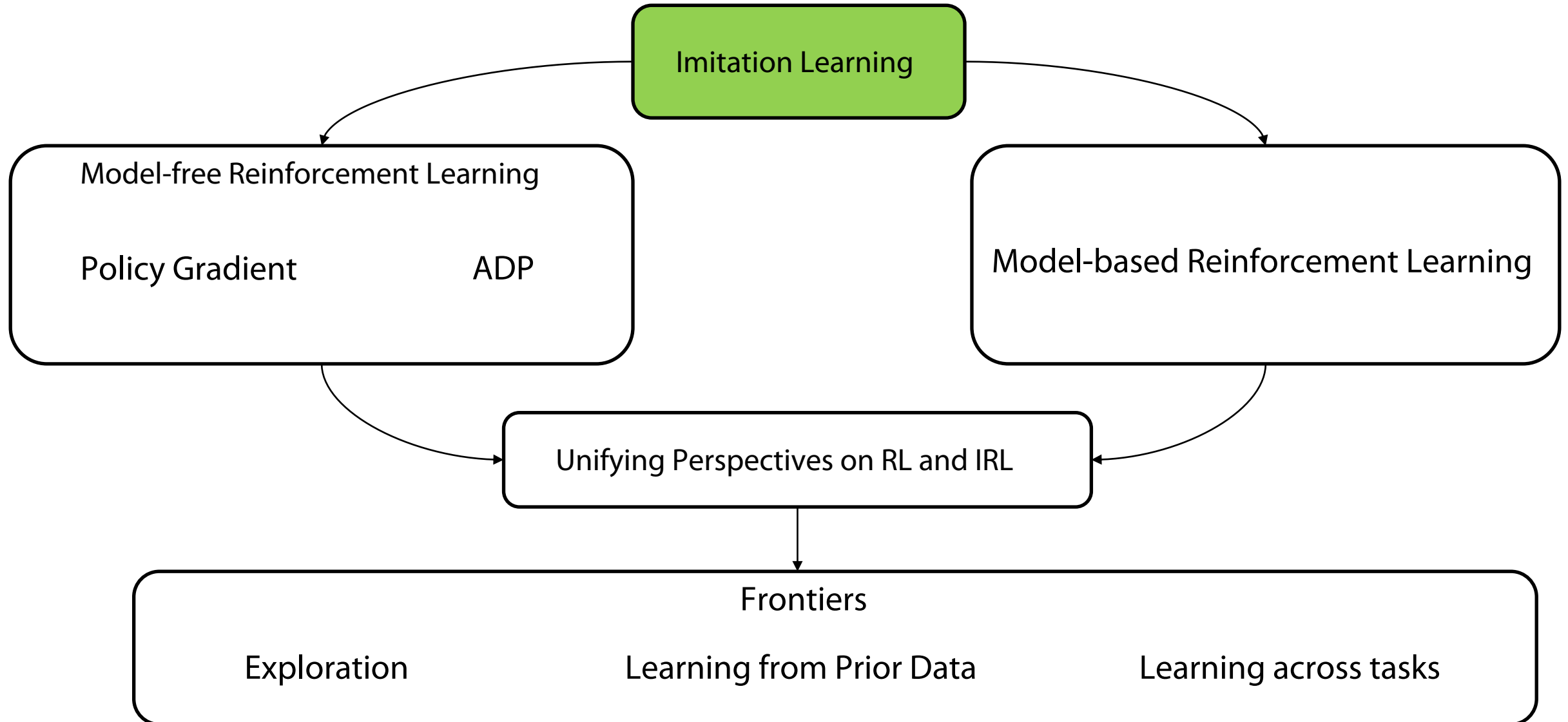


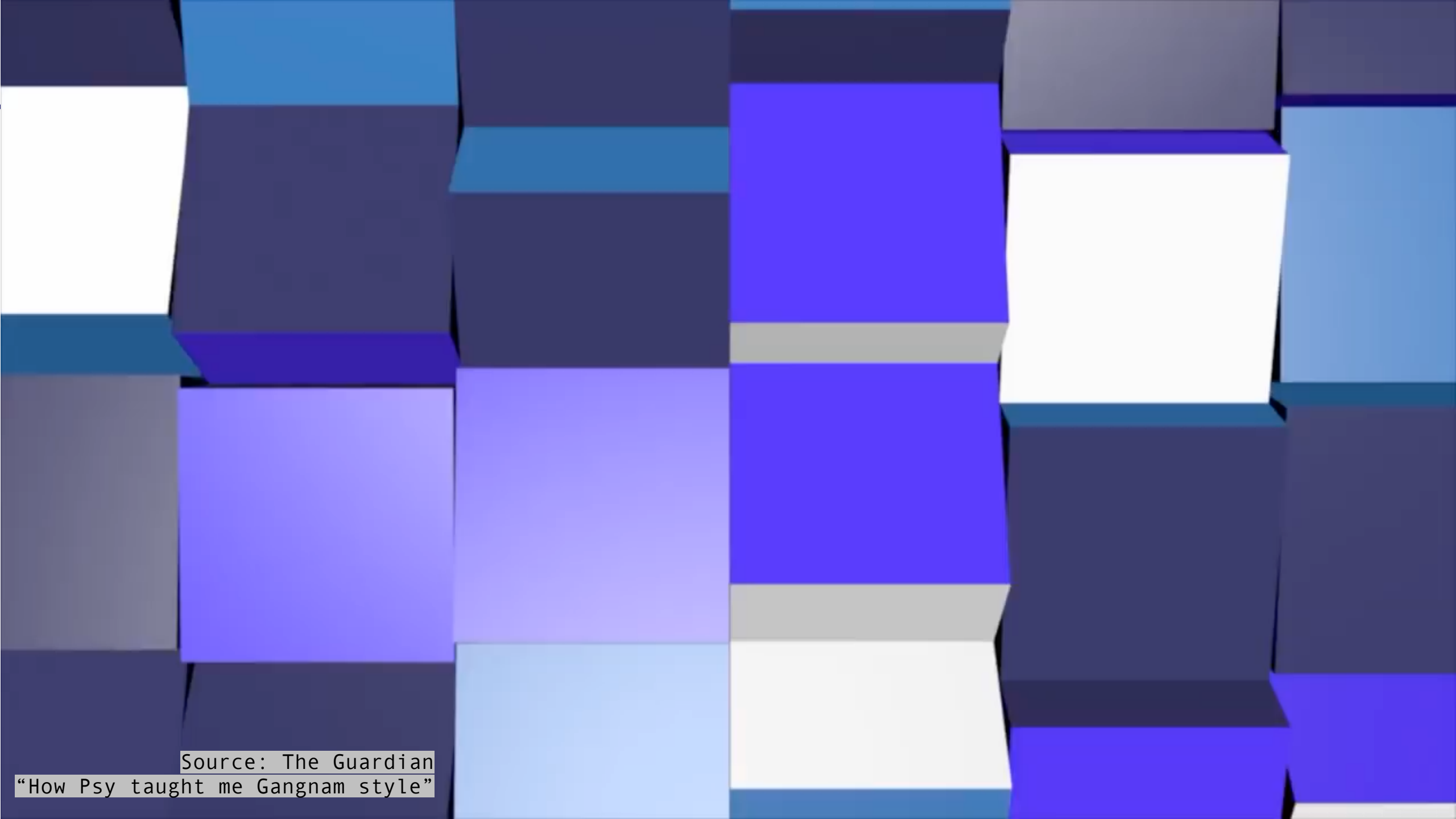
Multimodality and Underfitting in Imitation



Compounding Error in Imitation

Class Structure





Source: The Guardian

“How Psy taught me Gangnam style”

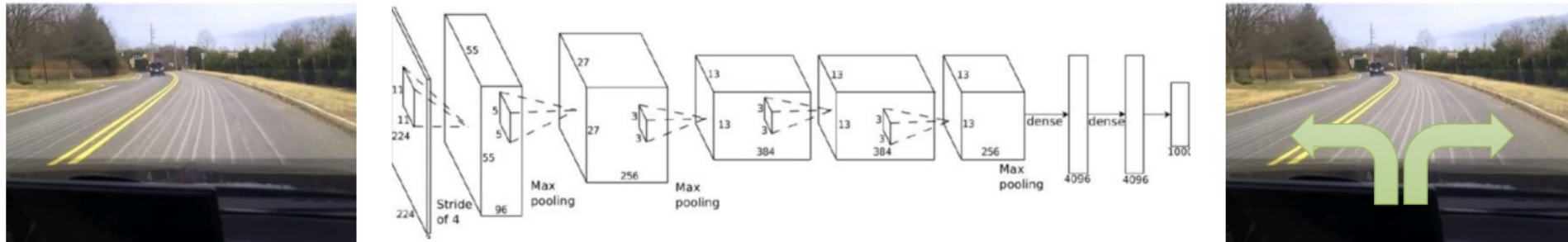
Imitation Learning: Intuition

Given: Demonstrations of optimal behavior

$$\mathcal{D} = \{s_0^i, a_0^i, s_1^i, a_1^i, \dots, s_T^i, a_T^i\}_{i=1}^N$$

Goal: Train a policy to mimic the demonstrator

$$\pi_{\theta}(a|s)$$

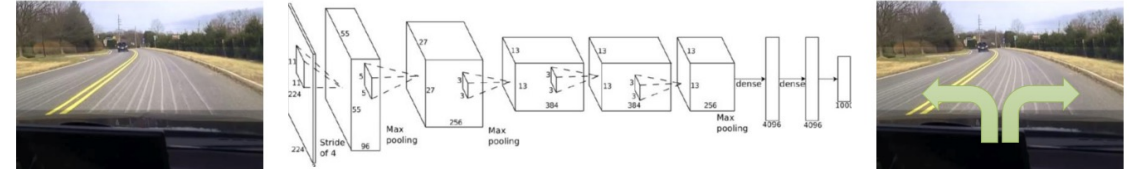


Pros: No rewards, online experience needed (?)

Why would we do this?

Given: Demonstrations of optimal behavior

Goal: Train a policy to mimic the demonstrator



Pros:

- ⊕ Avoids need for rewards, exploration
- ⊕ Natural way to do task specification
- ⊕ Can work well in practice

Cons:

- ⊖ Requires expert data, can be expensive
- ⊖ Cannot get better on deployment
- ⊖ Struggles on long horizon tasks

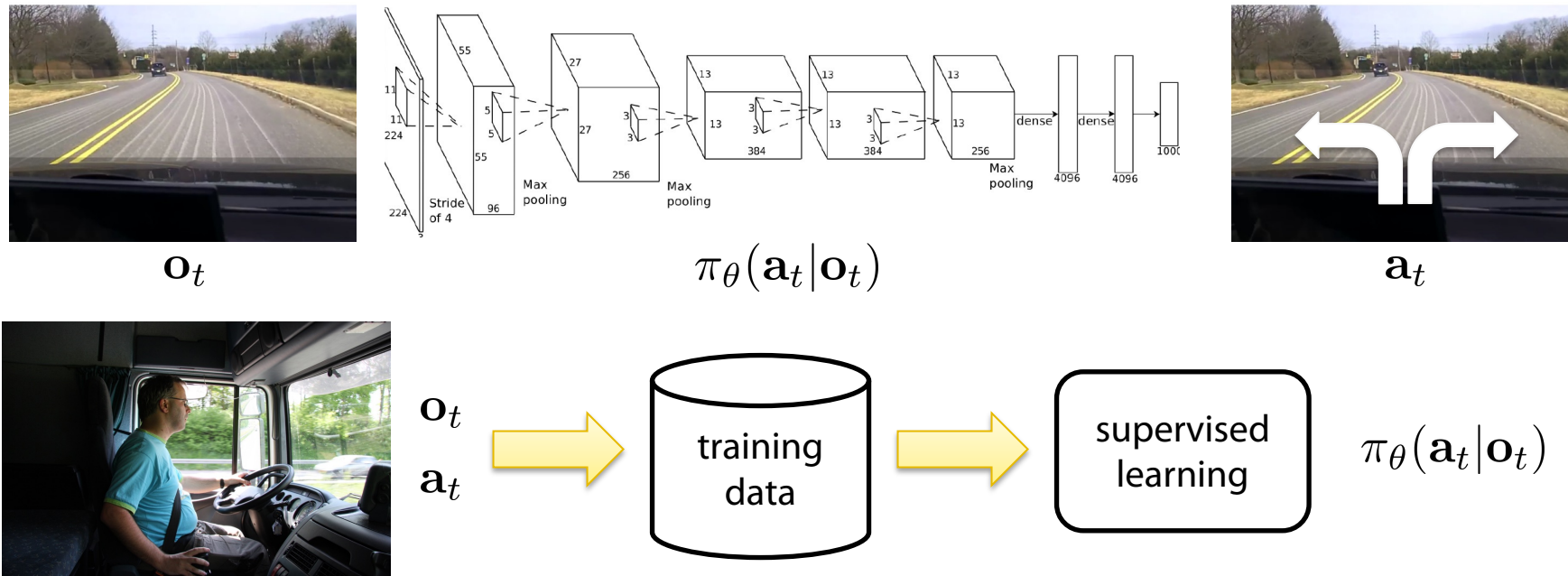
Idea 1: Imitation Learning via Supervised Learning

Given: Demonstrations of optimal behavior

$$\arg \max_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}} [\log \pi_{\theta}(a^* | s^*)]$$

Goal: Train a policy to mimic the demonstrator

Idea: Treat imitation learning as a supervised learning problem! → Behavior Cloning



Idea 1: Imitation Learning via Supervised Learning

Given: Demonstrations of optimal behavior

Goal: Train a policy to mimic the demonstrator

$$\arg \max_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}} [\log \pi_{\theta}(a^* | s^*)]$$

Discrete vs continuous

Maximum likelihood

```
if isinstance(env.action_space, gym.spaces.Box):
    criterion = nn.MSELoss()
else:
    criterion = nn.CrossEntropyLoss()
# Extract initial policy
model = student.policy.to(device)
def train(model, device, train_loader, optimizer):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        if isinstance(env.action_space, gym.spaces.Box):
            if isinstance(student, (A2C, PPO)):
                action, _, _ = model(data)
            else:
                action = model(data)
                action_prediction = action.double()
        else:
            dist = model.get_distribution(data)
            action_prediction = dist.distribution.logits
            target = target.long()
        loss = criterion(action_prediction, target)
        loss.backward()
        optimizer.step()
```

Idea 1: Imitation Learning via Supervised Learning

Given: Demonstrations of optimal behavior

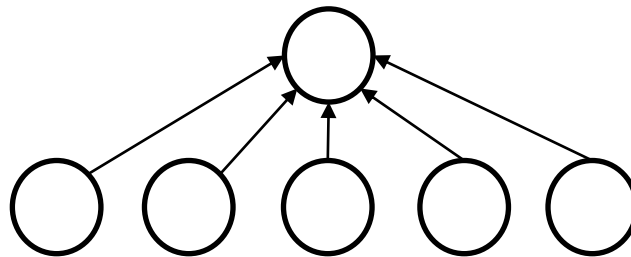
Goal: Train a policy to mimic the demonstrator

$$\arg \max_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}} [\log \pi_{\theta}(a^* | s^*)]$$

Tabular

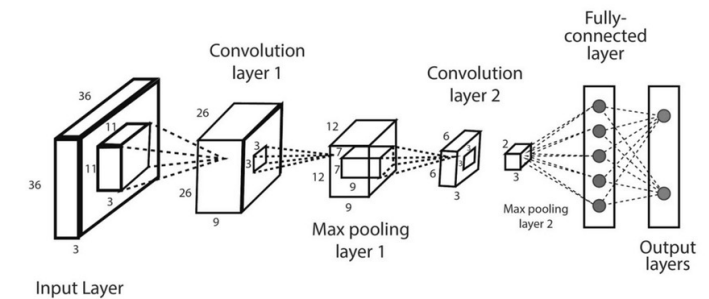
8.67	8.93	9.11	9.30	9.42
8.49		9.09	9.42	9.68
8.33		1.00		10.00
7.13	5.04	3.15	5.68	8.45
-10.00	-10.00	-10.00	-10.00	-10.00

Linear



$$\pi(a|s) = \langle \phi(s, a), w \rangle$$

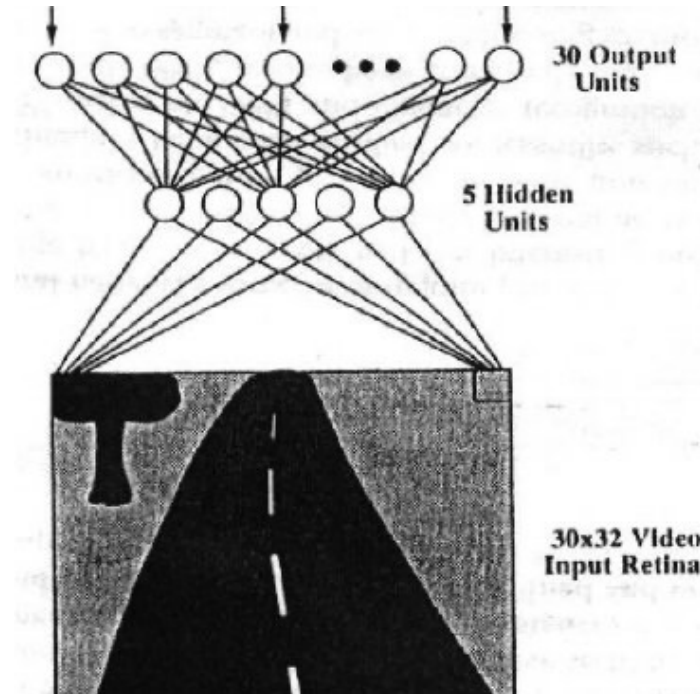
Arbitrary function approx



In practice, amounts to simple gradient based training with backpropagation

The original deep imitation learning system

ALVINN: **A**utonomous **L**and **V**ehicle **I**n a **N**eural **N**etwork
1989

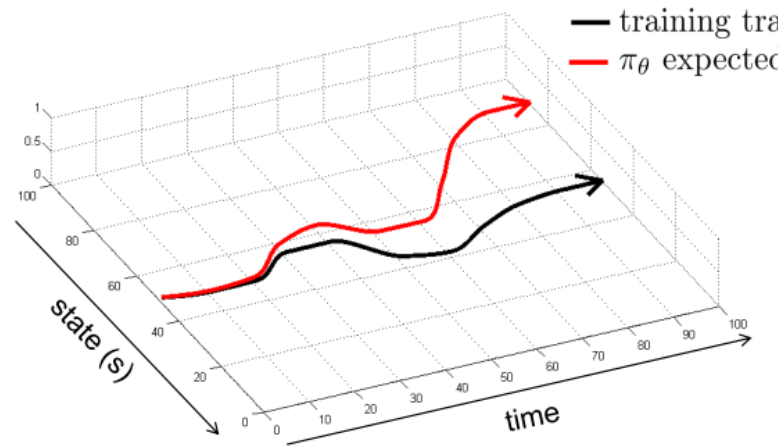


Where we are in 2024?



So does behavior cloning really work?

- Imitation Learning $\neq$ Supervised Learning



Compounding error!

$$\arg \max_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}} [\log \pi_{\theta}(a^* | s^*)]$$

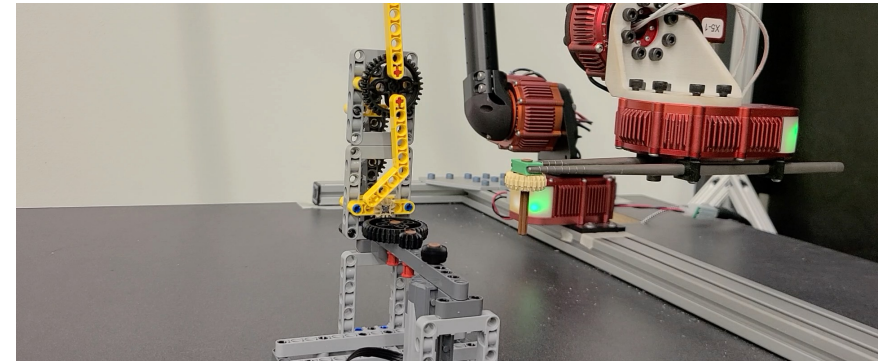
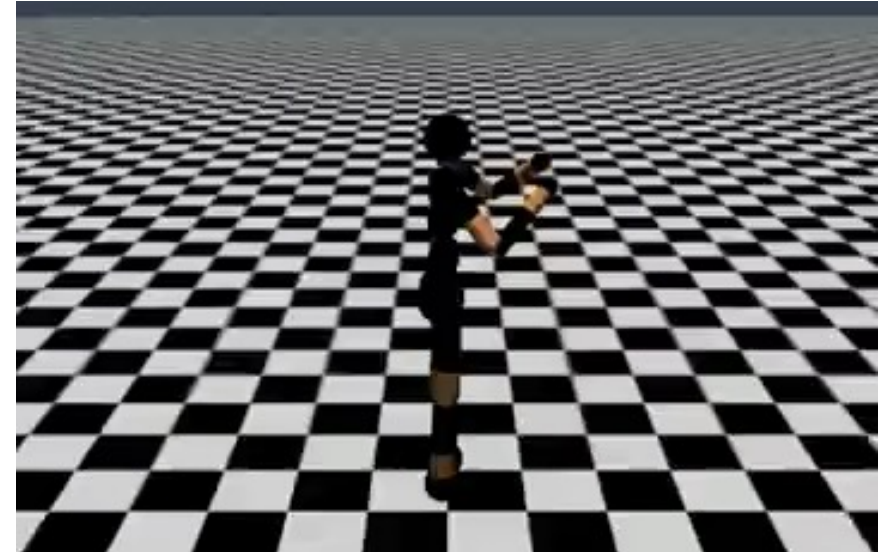
$$\mathbb{E}_{(s, a) \sim \rho(\pi)} [1(a = a^*)]$$



Not the same!

So does behavior cloning really work?

- Fails in practice as well!



What do we actually want?

- Imitation Learning can be formalized as matching the expert

(cost for generating an action
different than the expert)

$$c(s_t, a_t) = \begin{cases} 0, & \text{if } a_t = \pi^*(s_t), \\ 1, & \text{otherwise} \end{cases}$$

Measure deviation from expert
actions when the policy is rolled out

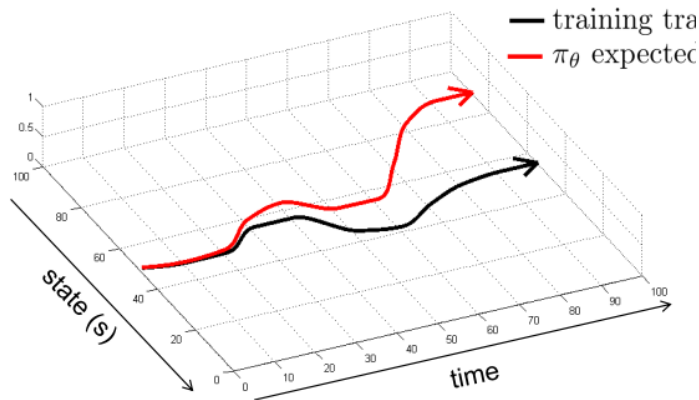
$$\mathbb{E}_{(s_t, a_t) \sim p_{\pi_\theta}(s_t, a_t)} [c(s_t, a_t)]$$

How bad is behavior cloning?

$$\arg \max_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}} [\log \pi_\theta(a^* | s^*)]$$

How well does BC do?: Intuition

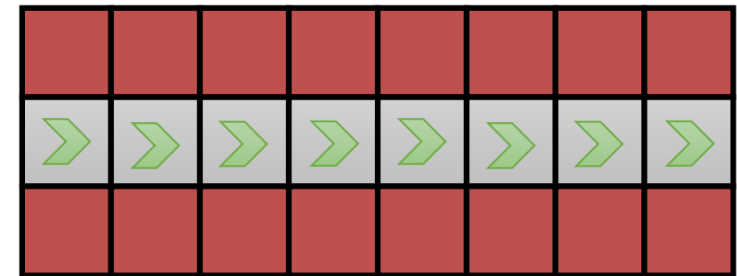
Behavior cloning has quadratically compounding error



$$\pi_\theta(a \neq \pi^*(s_t) | s_t) \leq \epsilon$$

Horizon H

If you fall off,
assume the worst



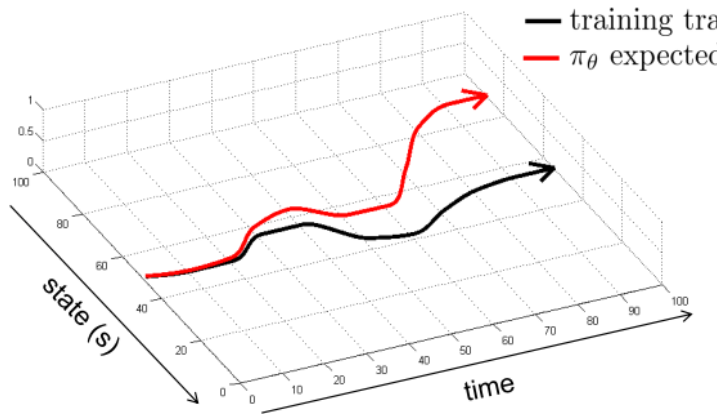
$$\underbrace{\mathbb{E} \left[\sum_t c(s_t, a_t) \right]}_{O(\epsilon H^2)} \leq \epsilon H + \dots + \dots$$

Union bound

Let's try and understand where the problem lies?

Behavior cloning has challenges in both theory and practice

$$\sum_t \mathbb{E}_{(s_t, a_t) \sim p_{\pi_\theta}(s_t, a_t)} [c(s_t, a_t)] \leq O(\epsilon H^2)$$



Underfitting

$$\pi_\theta(a \neq \pi^*(s_t) | s_t) \leq \epsilon$$

Compounding error

$$\leq O(\epsilon H^2)$$

Lecture outline

Recap: MDP formalism + why should we care?



Imitation learning: preliminaries and behavior cloning



Multimodality and Underfitting in Imitation

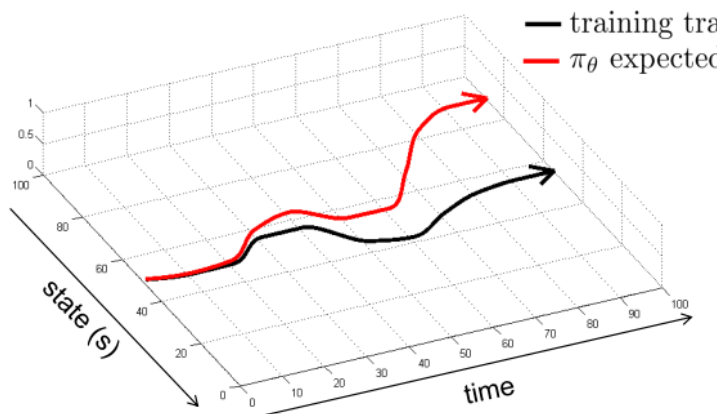


Compounding Error in Imitation

Let's try and understand where the problem lies?

Behavior cloning has challenges in both theory and practice

$$\sum_t \mathbb{E}_{(s_t, a_t) \sim p_{\pi_\theta}(s_t, a_t)} [c(s_t, a_t)] \leq O(\epsilon H^2)$$



Underfitting

$$\pi_\theta(a \neq \pi^*(s_t) | s_t) \leq \epsilon$$

Compounding error

$$\leq O(\epsilon H^2)$$

But won't a bigger neural net just solve this?

- Behavior cloning can underfit the data

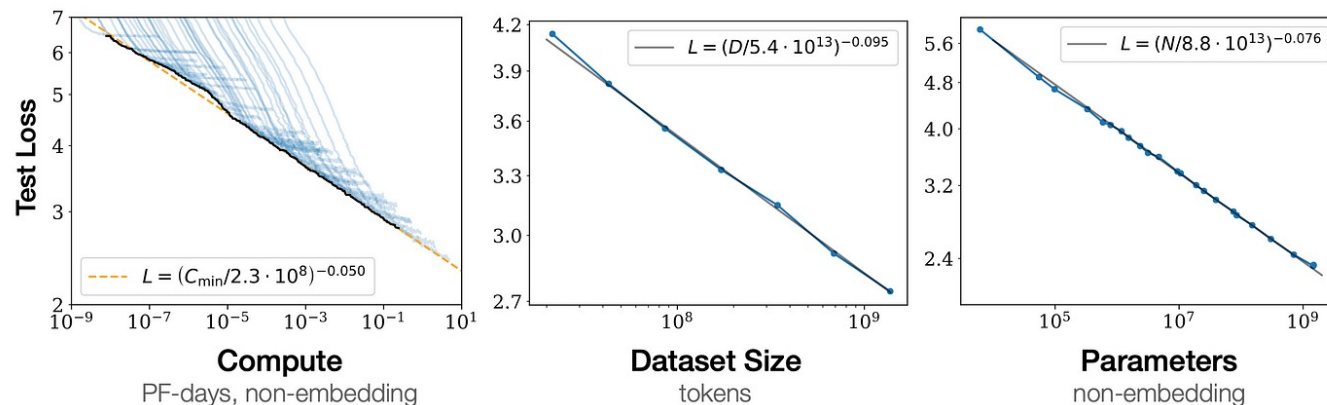
$$\sum_t \mathbb{E}_{(s_t, a_t) \sim p_{\pi_\theta}(s_t, a_t)} [c(s_t, a_t)] \leq O(\epsilon H^2)$$

$$\pi_\theta(a \neq \pi^*(s_t) | s_t) \leq \epsilon$$

for $s_t \sim p_{\text{train}}(s_t)$

May not be able to satisfy this

Q: won't a bigger model just solve the problem?

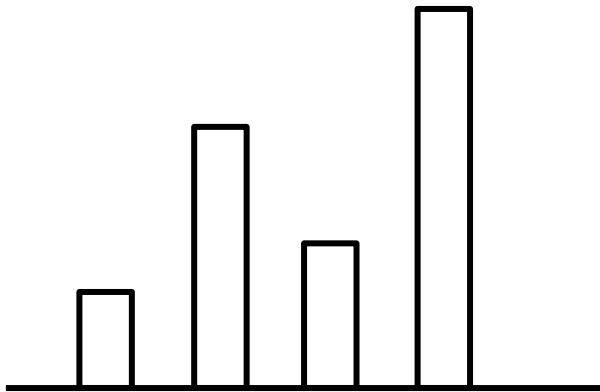


Kind of, but there's a fundamental problem!

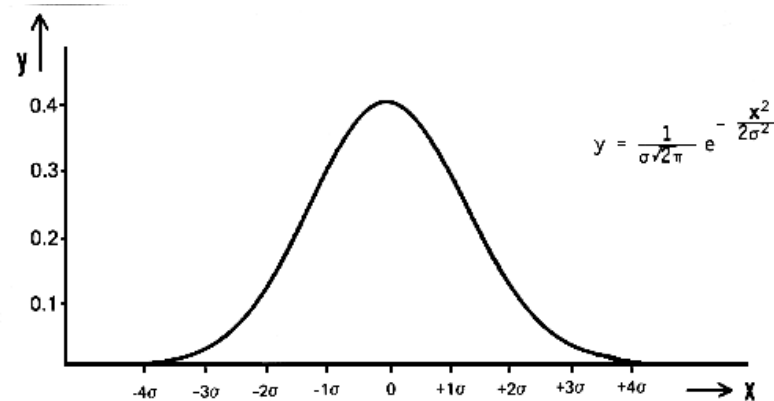
Distributional Expressivity

- Policy expressivity is a combination of expressivity of the function approximator and of the distribution family

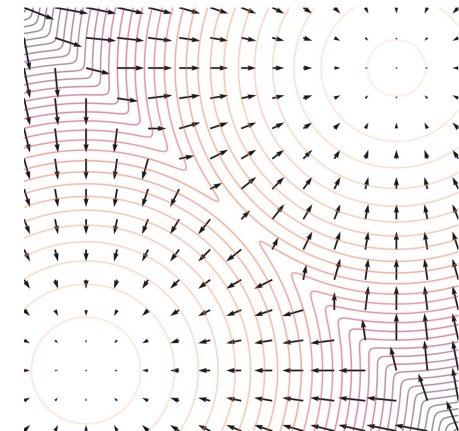
Categorical



Gaussian



Diffusion policy

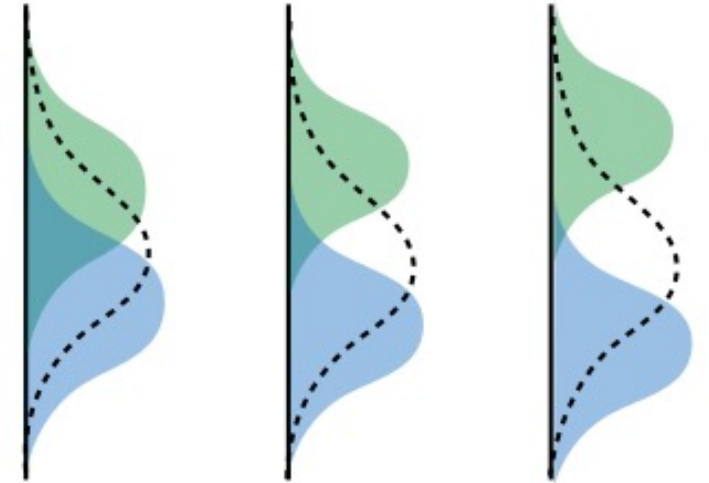
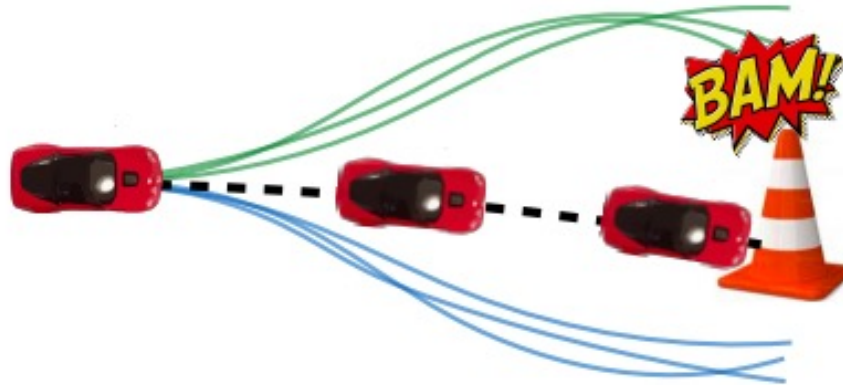
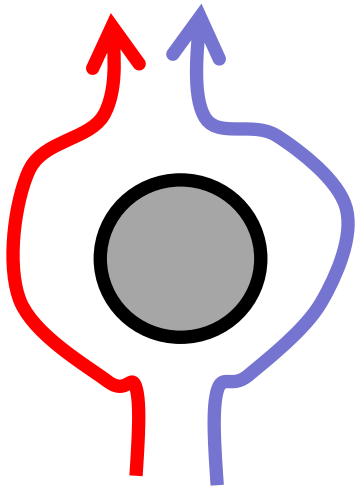


Tradeoff between expressivity and tractability

How does this reflect on imitation learning?

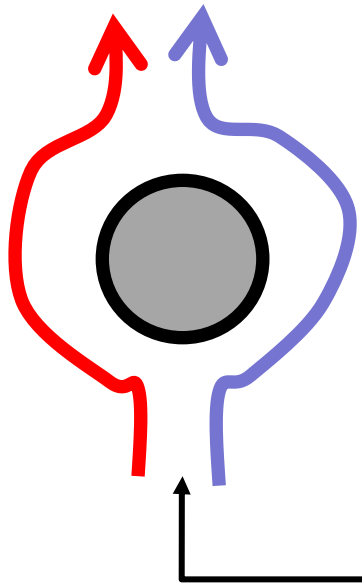
Let us consider a case with Gaussian policy

$$\arg \max_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}} [\log \pi_{\theta}(a^* | s^*)]$$



A combination of distributional expressivity and objective lead to mode averaging

Let's take a closer look at the objective



$$\arg \max_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}} [\log \pi_{\theta}(a^* | s^*)]$$

$$\max_{\theta} \mathbb{E}_{s^* \sim p_{\pi_e}(\cdot)} \left[\mathbb{E}_{a^* \sim \pi_e(\cdot | s^*)} [\log \pi_{\theta}(a^* | s^*) - \log \pi_e(a^* | s^*)] \right]$$

$$\min_{\theta} \mathbb{E}_{s^* \sim p_{\pi_e}(\cdot)} \left[\mathbb{E}_{a^* \sim \pi_e(\cdot | s^*)} \left[\log \frac{\pi_e(a^* | s^*)}{\pi_{\theta}(a^* | s^*)} \right] \right] = \mathbb{E}_{s^* \sim p_{\pi_e}(\cdot)} [D_{\text{KL}}(\pi_e(\cdot | s^*) || \pi_{\theta}(\cdot | s^*))]$$

Leads to mode averaging

Forward KL divergence

One instance of a broader class of divergences – f divergences $D_f(p(x), q(x)) = \mathbb{E}_{q(x)} \left[f \left(\frac{p(x)}{q(x)} \right) \right]$

Effects of choice of f-divergence on behavior

Different divergences lead to different properties

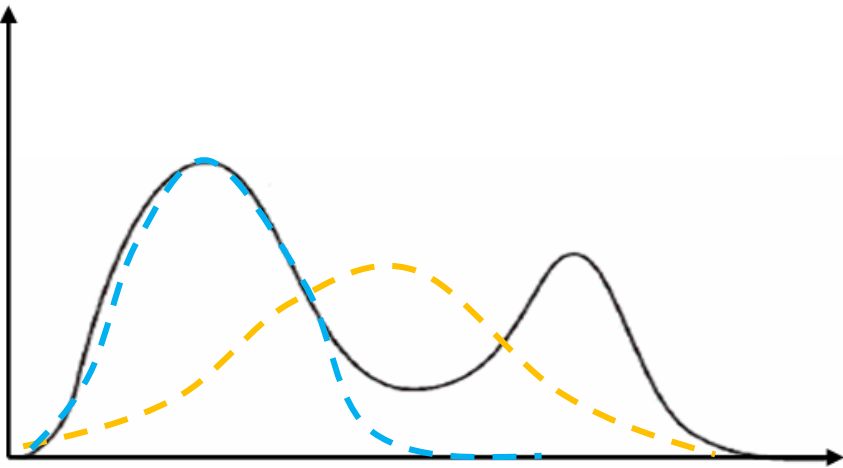
$$\mathbb{E}_{s^* \sim p_{\pi_e}(\cdot)} [D_{\text{KL}}(\pi_e(\cdot|s^*) || \pi_\theta(\cdot|s^*))] \longrightarrow \mathbb{E}_{s^* \sim p_{\pi_e}(\cdot)} [D_f(\pi_e(\cdot|s^*), \pi_\theta(\cdot|s^*))]$$

Forward KL (behavior cloning)

More general class of divergences

$$D_f(p(x), q(x)) = \mathbb{E}_{q(x)} \left[f \left(\frac{p(x)}{q(x)} \right) \right]$$

--- Forward KL (mode covering)	$f(x) = x \log(x)$
- - - Reverse KL (mode seeking)	$f(x) = -\log(x)$



So how do we fix BC?

Use a different f-divergence!
(Change f)

or

Use a richer distribution class!
(Change π_θ)

Using alternative f-divergences: Reverse KL

- Reverse KL helps, is mode seeking $D_{\text{RKL}}(\pi_e(.|s^*), \pi^\theta(.|s^*)) = \mathbb{E}_{\pi^\theta(.|s^*)} \left[\log \left(\frac{\pi^\theta(.|s^*)}{\pi_e(.|s^*)} \right) \right]$
- Challenge – requires known expert likelihood
- We need a sample based estimate!

Imitation Learning as f-Divergence Minimization

Liyiming Ke¹, Sanjiban Choudhury¹, Matt Barnes¹, Wen Sun², Gilwoo Lee¹,
and Siddhartha Srinivasa¹

Go read this!

$$\min_{\theta} \mathbb{E}_{\pi^\theta(.|s^*)} \left[\log \left(\frac{\pi^\theta(.|s^*)}{\pi_e(.|s^*)} \right) \right] \longleftrightarrow \min_{\theta} \max_{\phi} \mathbb{E}_{a \sim \pi^\theta(.|s^*)} [\phi(a)] - \mathbb{E}_{a \sim \pi_e(.|s^*)} [f^*(\phi(a))]$$

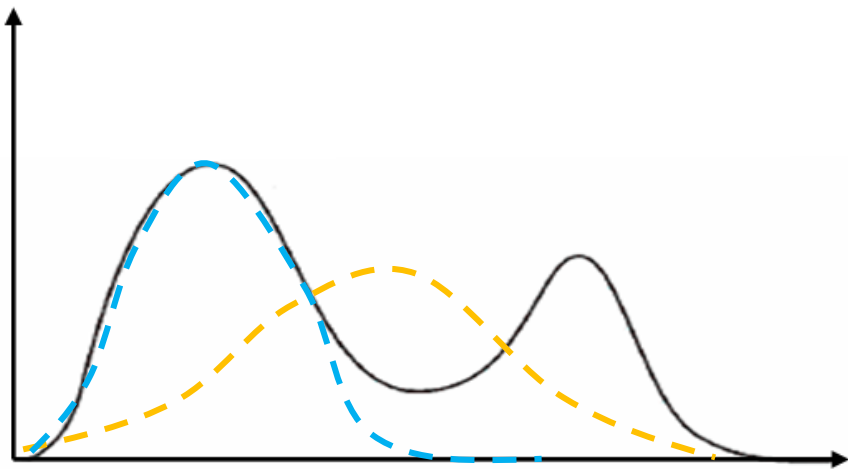
(Intractable) (Tractable – GAN style optimization)

Effects of choice of f-divergence on behavior

Different divergences lead to different properties

$$\mathbb{E}_{s^* \sim p_{\pi_e}(\cdot)} [D_{\text{KL}}(\pi_e(\cdot|s^*) || \pi_\theta(\cdot|s^*))] \longrightarrow \mathbb{E}_{s^* \sim p_{\pi_e}(\cdot)} [D_f(\pi_e(\cdot|s^*), \pi_\theta(\cdot|s^*))]$$

Forward KL (behavior cloning)



More general class of divergences

$$D_f(p(x), q(x)) = \mathbb{E}_{q(x)} \left[f \left(\frac{p(x)}{q(x)} \right) \right]$$

--- Forward KL (mode covering)	$f(x) = x \log(x)$
--- Reverse KL (mode seeking)	$f(x) = -\log(x)$

So how do we fix BC?

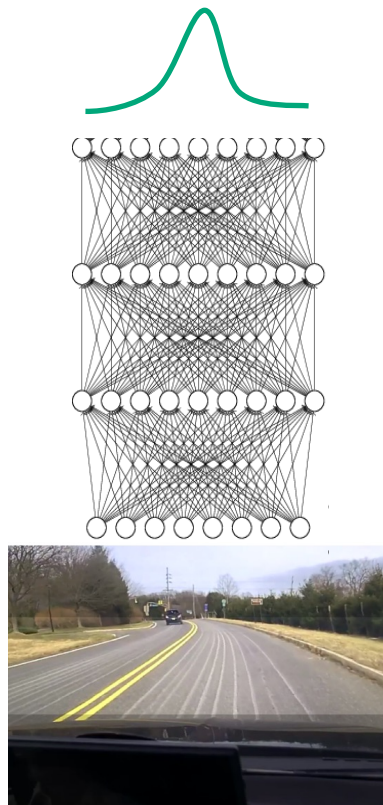
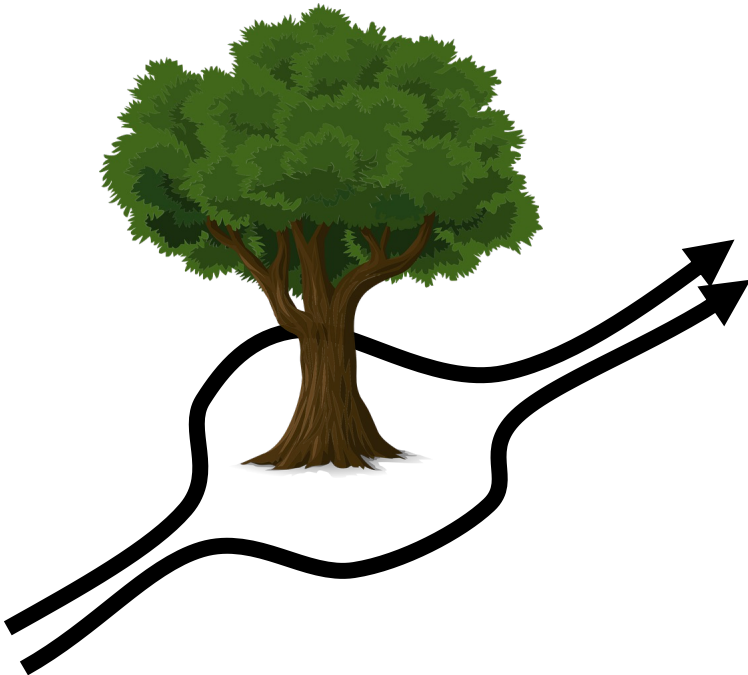
Use a different f-divergence!
(Change f)

or

Use a richer distribution class!
(Change π_θ)

Using Richer Policy Distribution Classes

Multimodal behavior → use more **expressive** probability distributions, no mode averaging issues



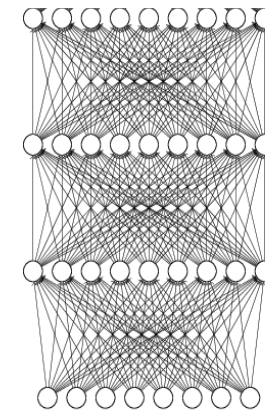
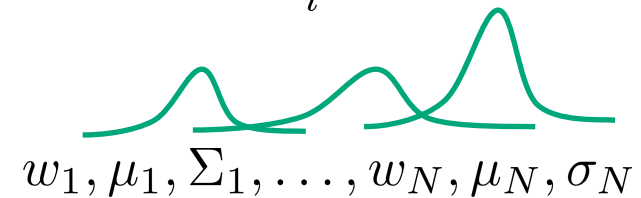
1. Output mixture of Gaussians
2. Latent variable models
3. Autoregressive discretization
4. Diffusion models
5. ...



Why might we fail to fit the expert?

- ➔
1. Output mixture of Gaussians
 2. Latent variable models
 3. Autoregressive discretization
 4. Diffusion models
 5. ...

$$\pi(\mathbf{a}|\mathbf{o}) = \sum_i w_i \mathcal{N}(\mu_i, \Sigma_i)$$



Why might we fail to fit the expert?

1. Output mixture of Gaussians
2. Latent variable models
- ➔ 3. Autoregressive discretization
4. Diffusion models
5. ...

Why does this work?

first step: $p(a_{t,0}|\mathbf{s}_t)$

second step: $p(a_{t,1}|\mathbf{s}_t, a_{t,0})$

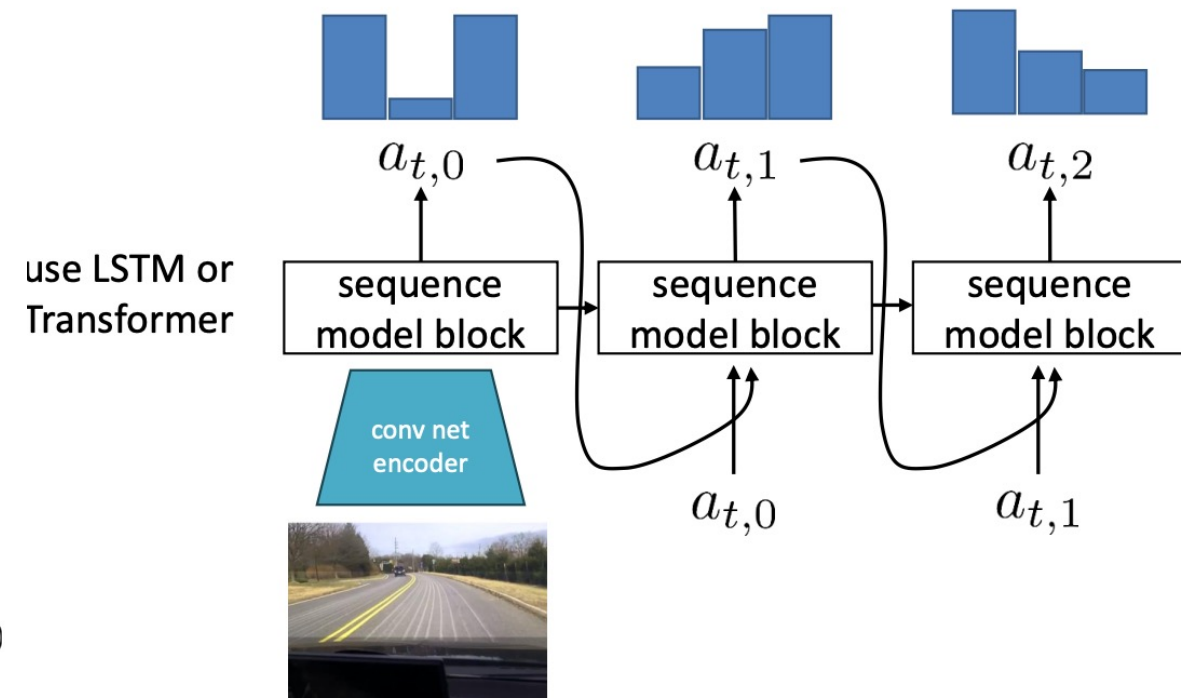
third step: $p(a_{t,2}|\mathbf{s}_t, a_{t,0}, a_{t,1})$

$$p(a_{t,2}|\mathbf{s}_t, a_{t,0}, a_{t,1})p(a_{t,1}|\mathbf{s}_t, a_{t,0})p(a_{t,0}|\mathbf{s}_t)$$

$$= p(a_{t,0}, a_{t,1}, a_{t,2}|\mathbf{s}_t)$$

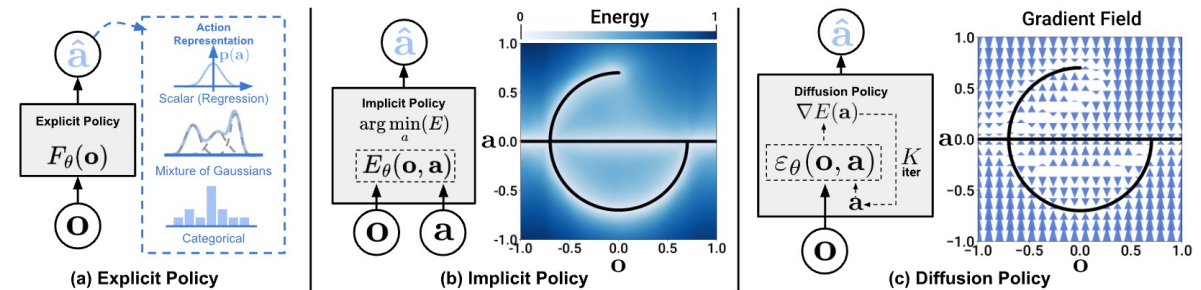
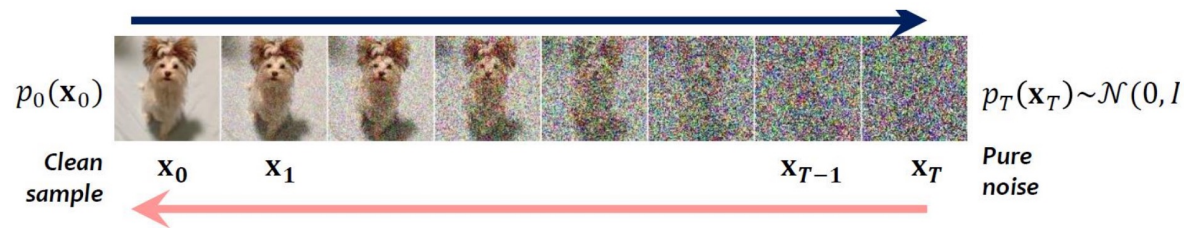
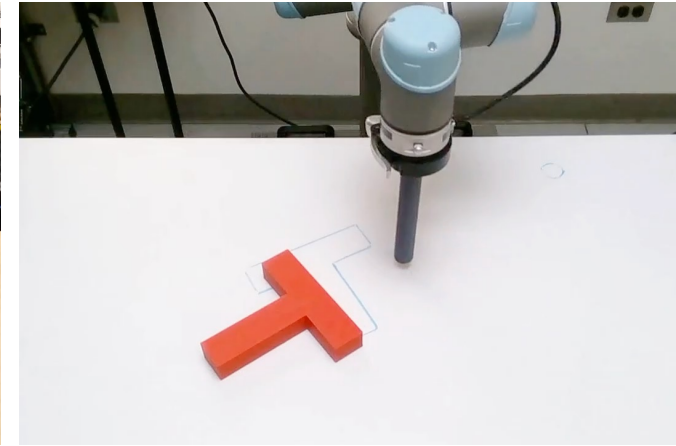
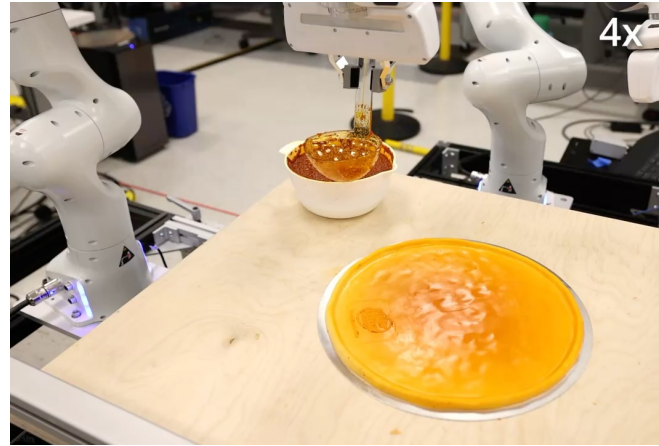
$$= p(\mathbf{a}_t|\mathbf{s}_t)$$

$$\mathbf{a}_t = \begin{pmatrix} 0.1 \\ 1.2 \\ -0.3 \end{pmatrix} \begin{matrix} a_{t,0} \\ a_{t,1} \\ a_{t,2} \end{matrix}$$



Why might we fail to fit the expert?

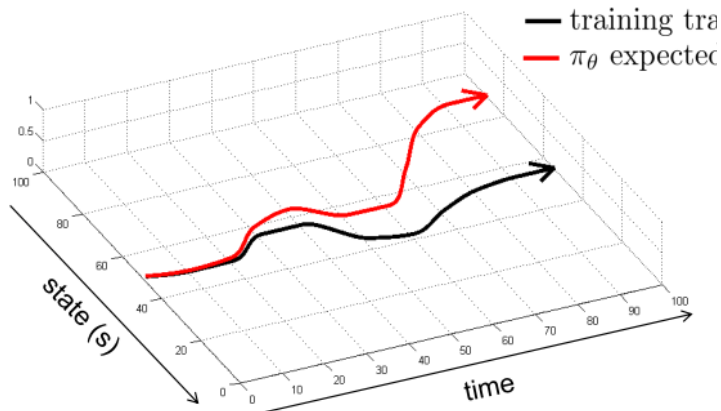
1. Output mixture of Gaussians
2. Latent variable models
3. Autoregressive discretization
- ➔ 4. Diffusion models
5. ...



Let's try and understand where the problem lies?

Behavior cloning has challenges in both theory and practice

$$\sum_t \mathbb{E}_{(s_t, a_t) \sim p_{\pi_\theta}(s_t, a_t)} [c(s_t, a_t)] \leq O(\epsilon H^2)$$



Underfitting

$$\pi_\theta(a \neq \pi^*(s_t) | s_t) \leq \epsilon$$

Compounding error

$$\leq O(\epsilon H^2)$$

Richer policy class Alternative Divergence

Lecture outline

Recap: MDP formalism + why should we care?



Imitation learning: preliminaries and behavior cloning



Multimodality and Underfitting in Imitation

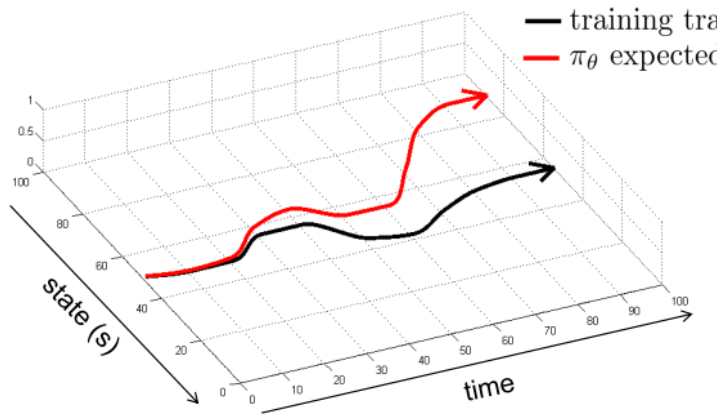


Compounding Error in Imitation

Let's try and understand where the problem lies?

Behavior cloning has challenges in both theory and practice

$$\sum_t \mathbb{E}_{(s_t, a_t) \sim p_{\pi_\theta}(s_t, a_t)} [c(s_t, a_t)] \leq O(\epsilon H^2)$$



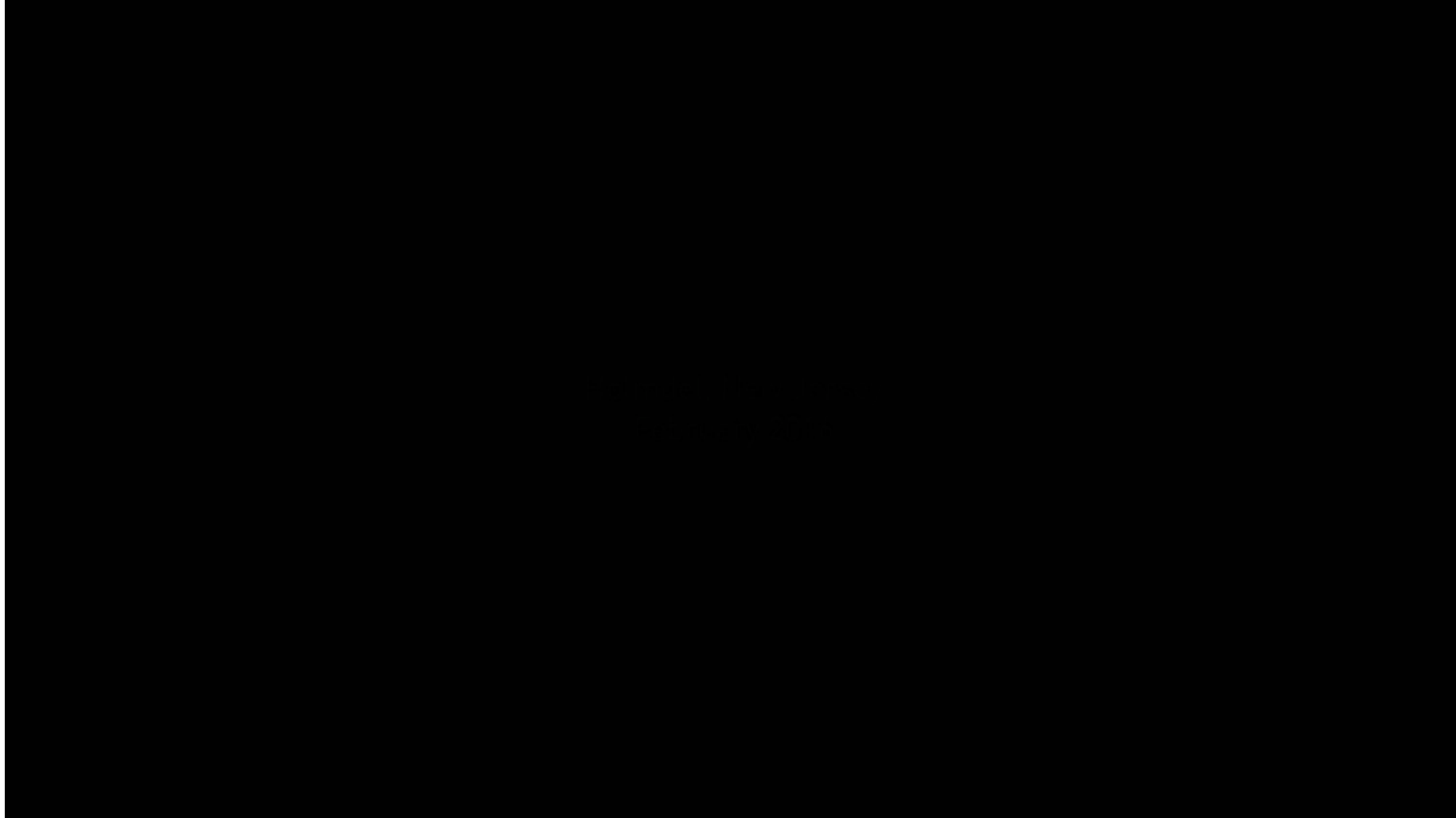
Underfitting

$$\pi_\theta(a \neq \pi^*(s_t) | s_t) \leq \epsilon$$

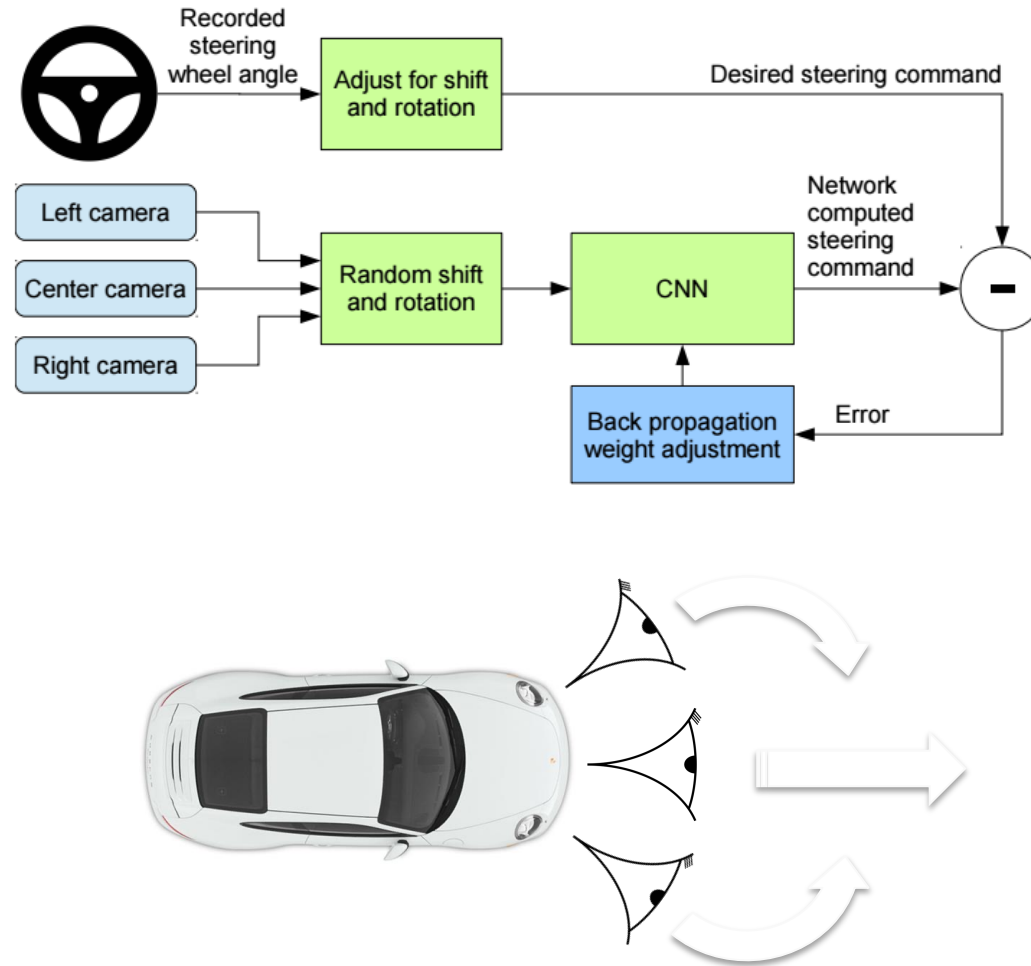
Compounding error

$$\leq O(\epsilon H^2)$$

Can we avoid compounding error in special cases?

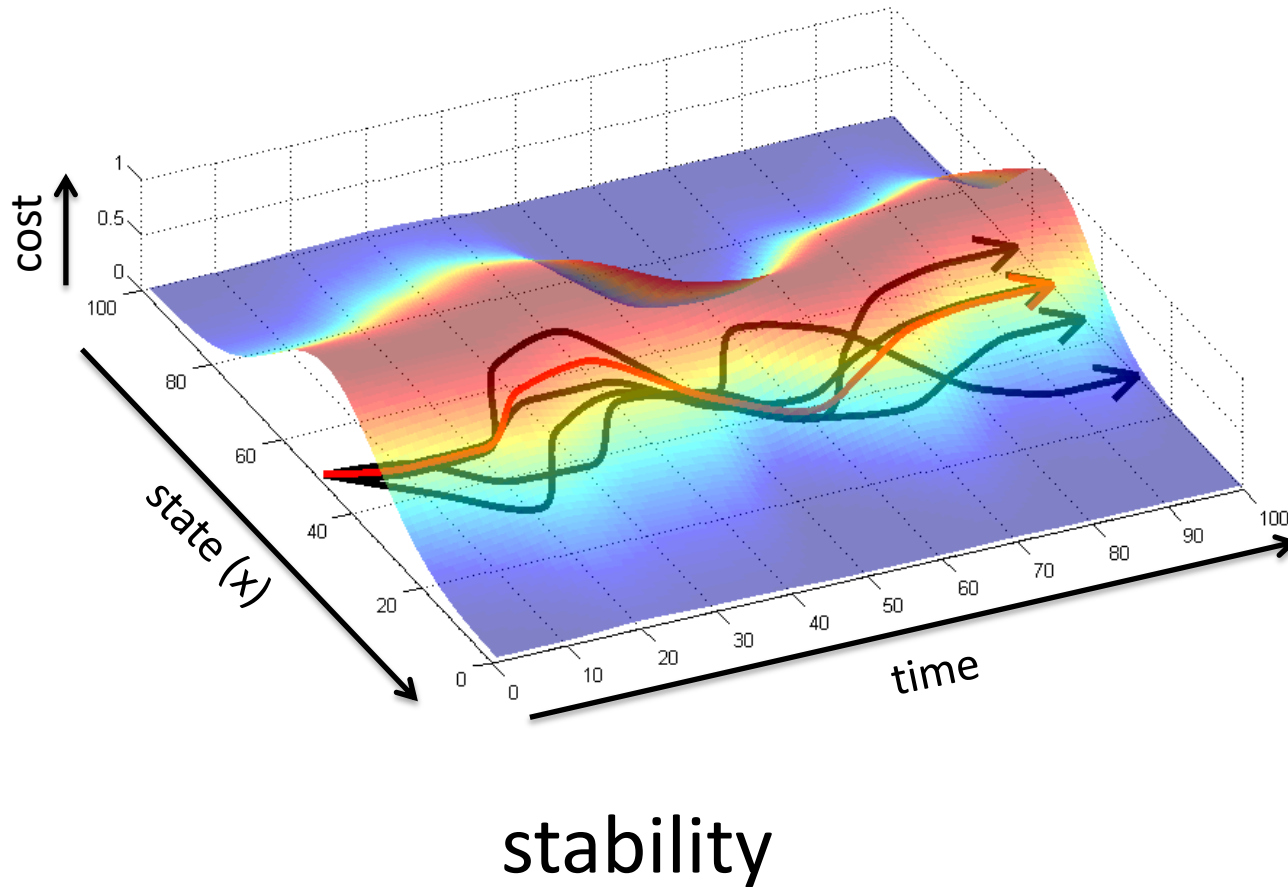


Why did that work?

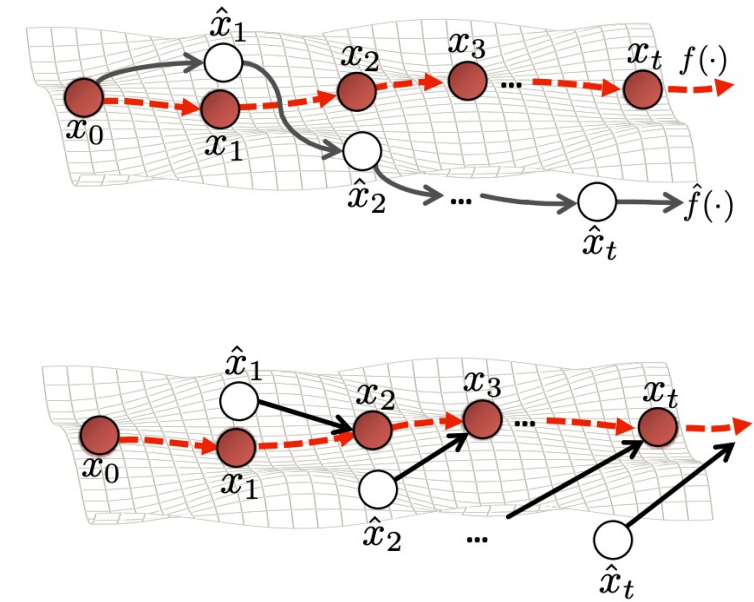


What is the general principle?

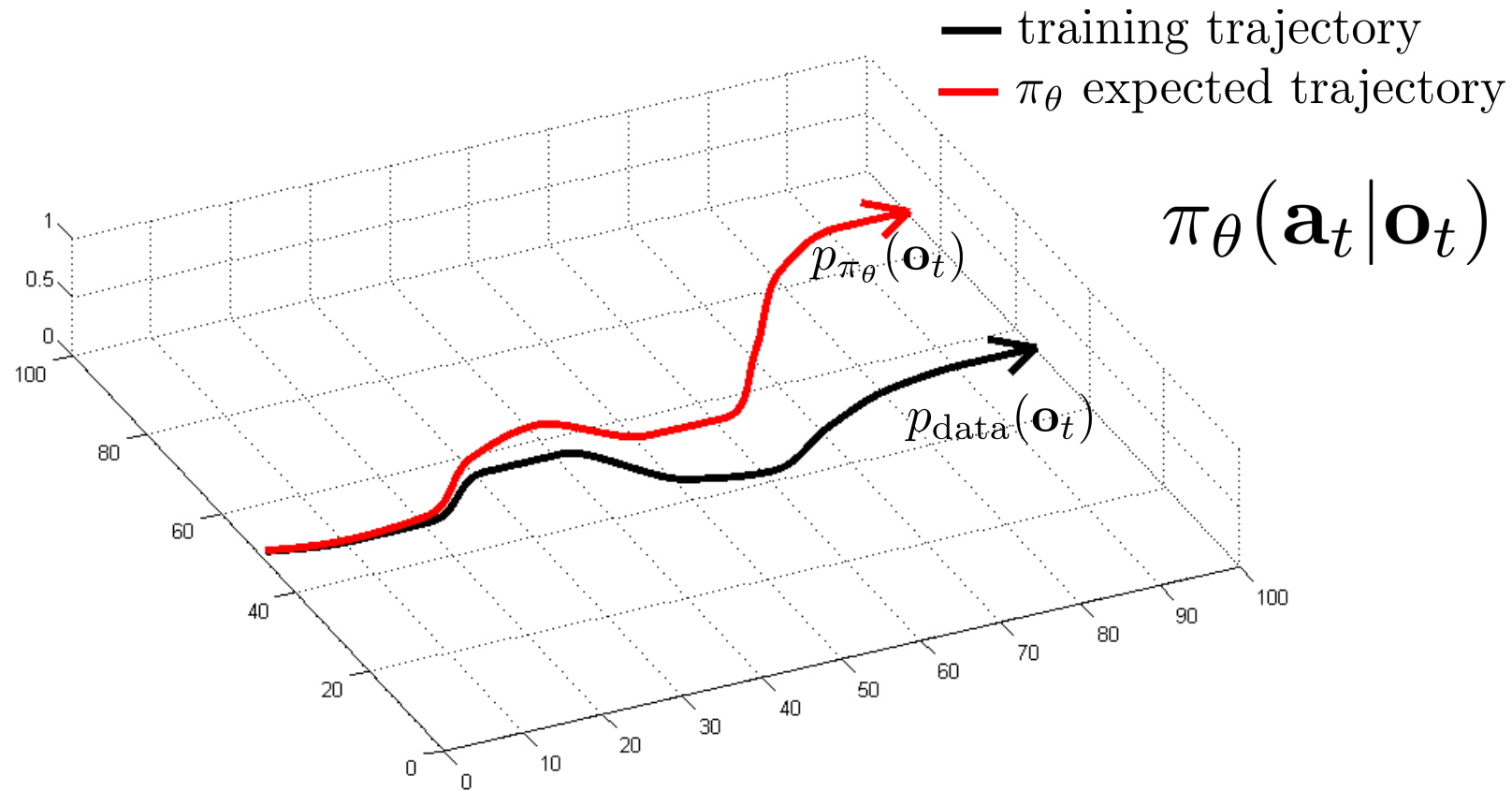
- training trajectory
- π_θ expected trajectory



Corrective labels that bring you back to the data



What might this mean mathematically?



can we make $p_{\text{data}}(\mathbf{o}_t) = p_{\pi_\theta}(\mathbf{o}_t)$?

Concrete Instantiation: DAgger

can we make $p_{\text{data}}(\mathbf{o}_t) = p_{\pi_\theta}(\mathbf{o}_t)$?

idea: instead of being clever about $p_{\pi_\theta}(\mathbf{o}_t)$, be clever about $p_{\text{data}}(\mathbf{o}_t)$!

DAgger: Dataset Aggregation

goal: collect training data from $p_{\pi_\theta}(\mathbf{o}_t)$ instead of $p_{\text{data}}(\mathbf{o}_t)$

how? just run $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

but need labels $\mathbf{a}_t$!

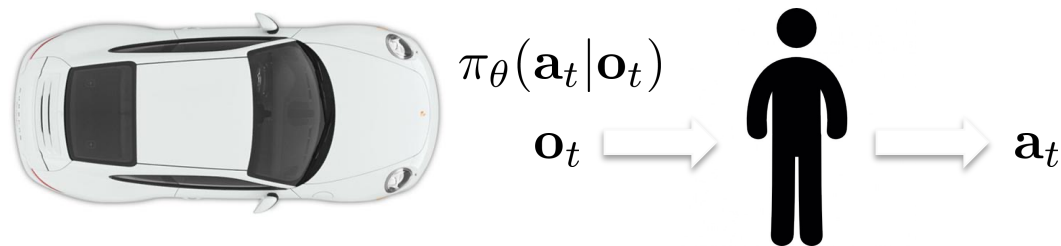
- 
1. train $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ from human data $\mathcal{D} = \{\mathbf{o}_1, \mathbf{a}_1, \dots, \mathbf{o}_N, \mathbf{a}_N\}$
 2. run $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ to get dataset $\mathcal{D}_\pi = \{\mathbf{o}_1, \dots, \mathbf{o}_M\}$
 3. Ask human to label $\mathcal{D}_\pi$ with actions $\mathbf{a}_t$
 4. Aggregate: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_\pi$

DAgger Example

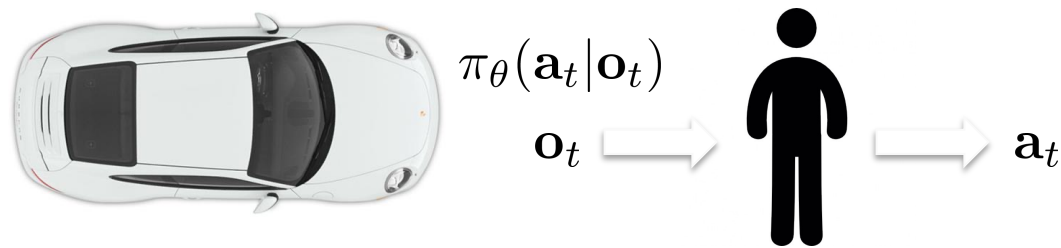
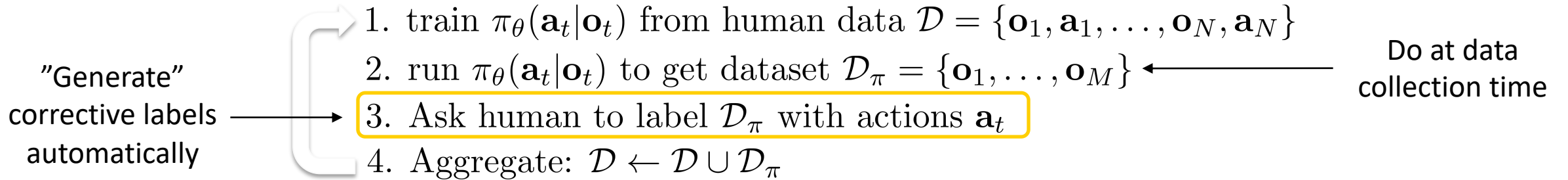


What's the problem?

1. train $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$ from human data $\mathcal{D} = \{\mathbf{o}_1, \mathbf{a}_1, \dots, \mathbf{o}_N, \mathbf{a}_N\}$
2. run $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$ to get dataset $\mathcal{D}_{\pi} = \{\mathbf{o}_1, \dots, \mathbf{o}_M\}$
3. Ask human to label $\mathcal{D}_{\pi}$ with actions $\mathbf{a}_t$
4. Aggregate: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_{\pi}$

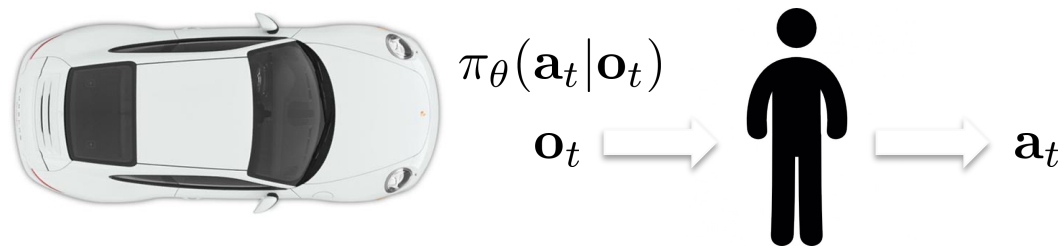


How might we fix this?



How might we fix this?

1. train $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ from human data $\mathcal{D} = \{\mathbf{o}_1, \mathbf{a}_1, \dots, \mathbf{o}_N, \mathbf{a}_N\}$
 2. run $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ to get dataset $\mathcal{D}_\pi = \{\mathbf{o}_1, \dots, \mathbf{o}_M\}$ ←
 3. Ask human to label $\mathcal{D}_\pi$ with actions $\mathbf{a}_t$
 4. Aggregate: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_\pi$
- Do at data collection time**



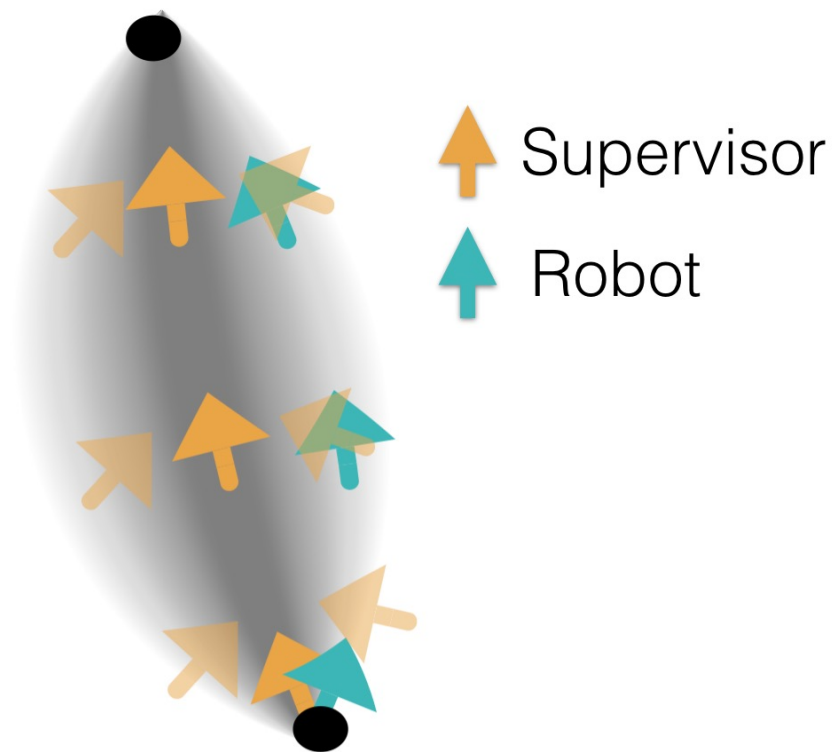
Noising the Data Collection Process

Key idea: force the human to correct for noise **during** training

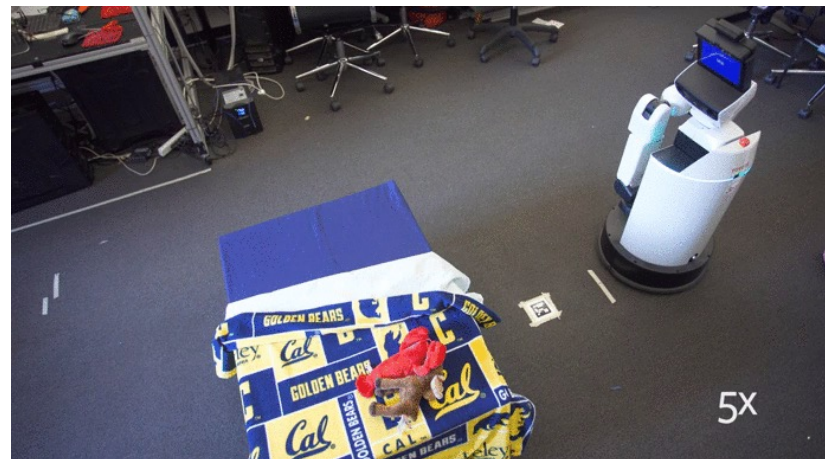
Under noise during data collection

Maximize likelihood

$$\hat{\psi}_{k+1} = \underset{\psi}{\operatorname{argmin}} E_{p(\xi|\pi_{\theta^*}, \psi_k)} - \sum_{t=0}^{T-1} \log [\pi_{\theta^*}(\pi_{\hat{\theta}}(\mathbf{x}_t)|\mathbf{x}_t, \psi)]$$

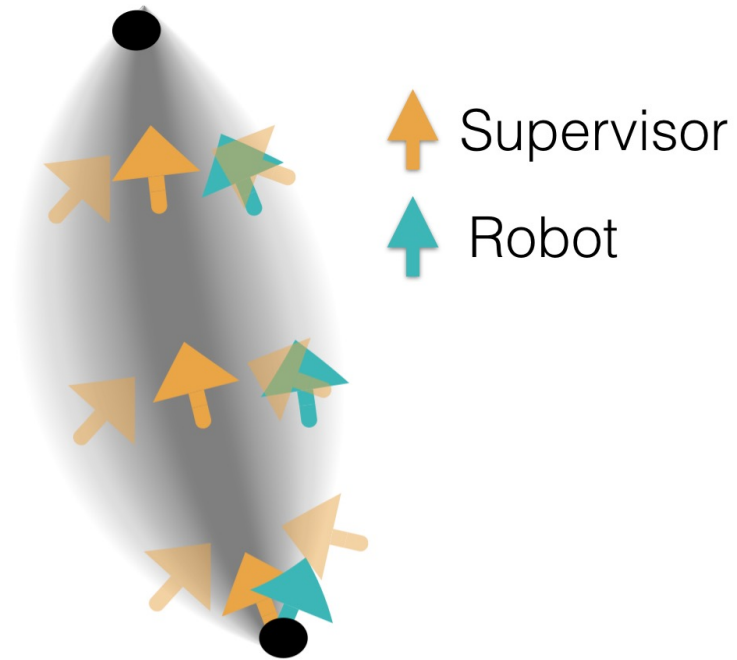


Noise Injection



Why might this not be enough?

Key idea: force the human to correct for noise during training



Noise Injection

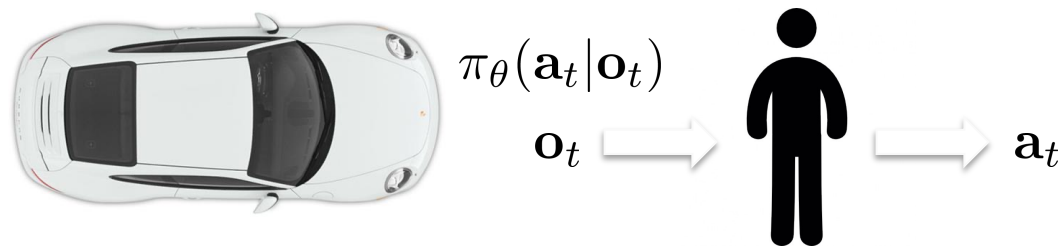


Assumes that the expert can actually perform behaviors under noise
→ Not always possible!

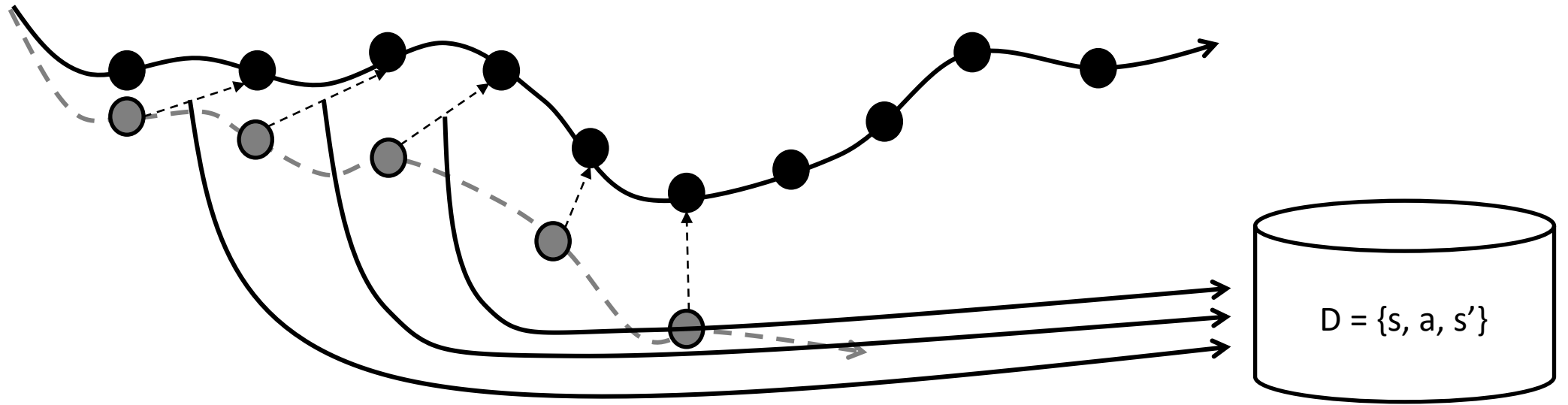
How might we fix this?

"Generate"
corrective labels
automatically

1. train $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$ from human data $\mathcal{D} = \{\mathbf{o}_1, \mathbf{a}_1, \dots, \mathbf{o}_N, \mathbf{a}_N\}$
2. run $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$ to get dataset $\mathcal{D}_{\pi} = \{\mathbf{o}_1, \dots, \mathbf{o}_M\}$
3. Ask human to label $\mathcal{D}_{\pi}$ with actions $\mathbf{a}_t$
4. Aggregate: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_{\pi}$



Can we avoid expensive online data collection/labeling?



Generate corrective labels
to dataset for imitation

How can we find corrective labels without an expensive human in the loop
and online data collection?



Abhay
Deshpande

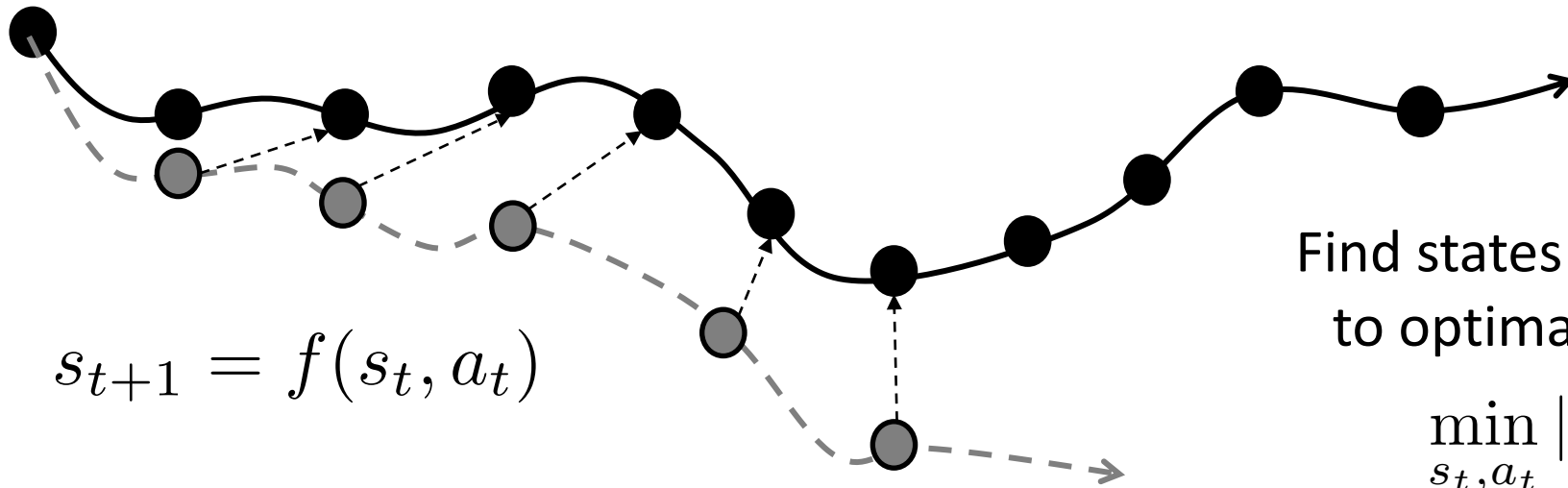


Yunchu
Zhang



Liyiming
Ke

Generating Corrective Labels From True Dynamics



Find states (s_t), actions (a_t) that lead back to optimal states under true dynamics

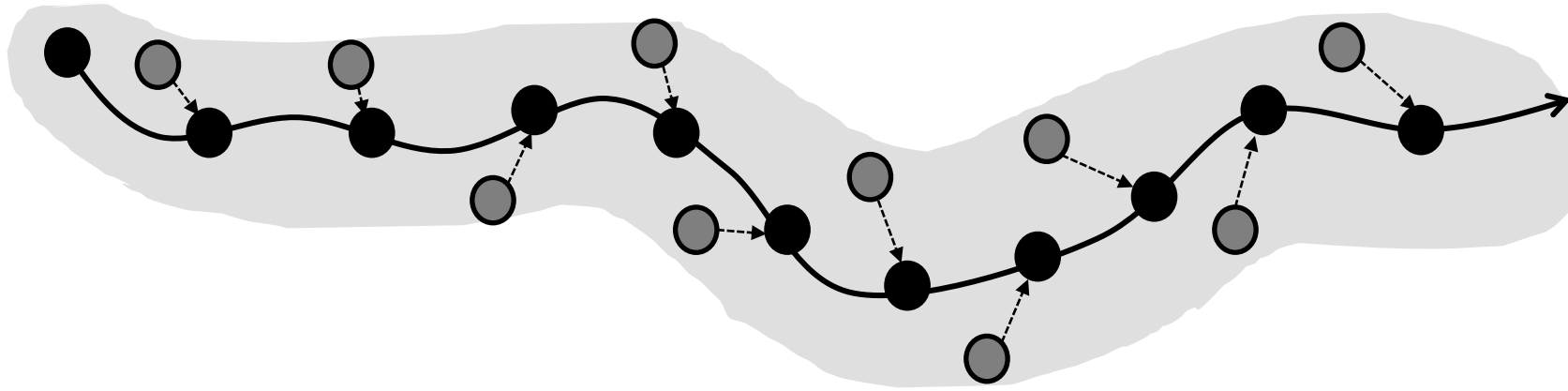
$$\min_{s_t, a_t} \|s_{t+1}^* - f(s_t, a_t)\| \leq \epsilon$$

Easy with known dynamics

Intuition: find labels to bring OOD states back in distribution

But models are unknown! ☹️

Generating Corrective Labels with Learned Dynamics



Ok models are unknown,
let's learn them!

$$\min_{\hat{f}} \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left[\|\hat{f}(s_t, a_t) - s_{t+1}\|_2 \right]$$

But learned dynamics $\hat{f}_\phi$ are not globally accurate?



Under approximately Lipschitz smooth
models, trust models around training data

$$\|s_{t+1}^* - \hat{f}_\phi(s_t, a_t)\| \leq \epsilon$$

Find states (s_t), actions (a_t) that lead back to
optimal states under ~~true~~ learned dynamics,
where learned dynamics can be trusted

$$\min_{s_t, a_t} \|s_{t+1}^* - \hat{f}_\phi(s_t, a_t)\| \leq \epsilon \longleftarrow \text{Corrective label}$$

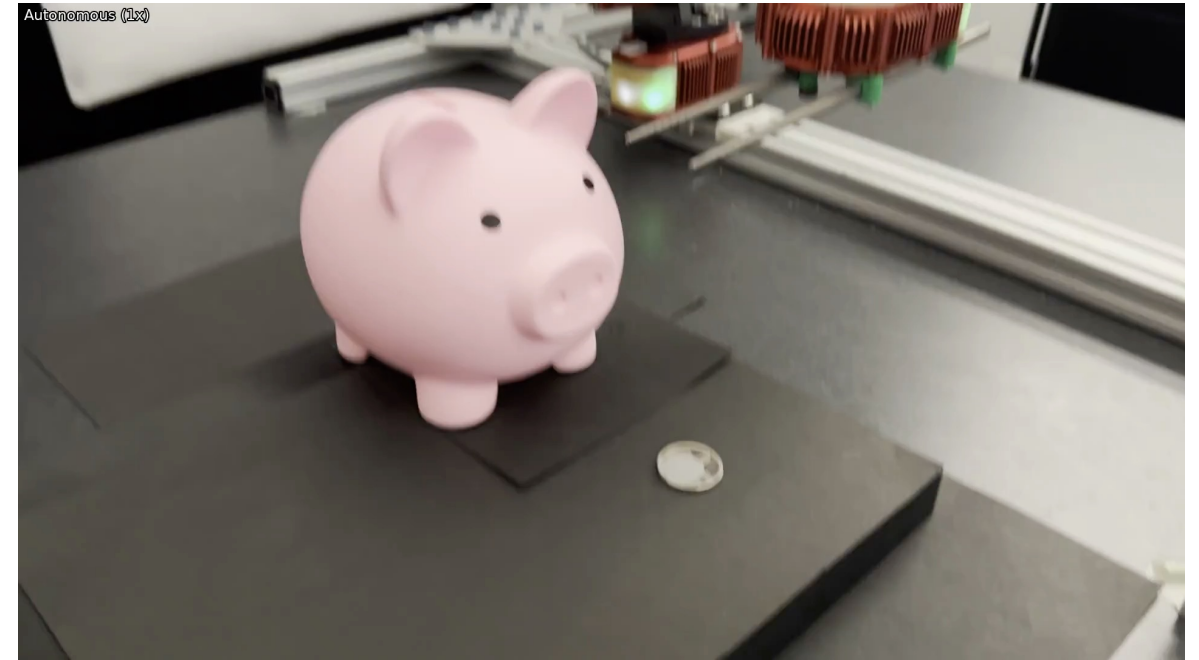
$$\text{s.t. } \|s_t^* - s_t\| \leq \epsilon_1, \|a_t^* - a_t\| \leq \epsilon_2 \longleftarrow \text{Close to data}$$

How well does generating corrective labels work?

With corrective labels

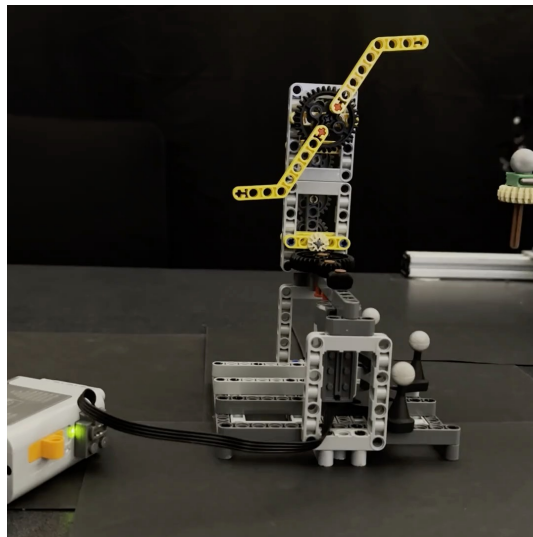
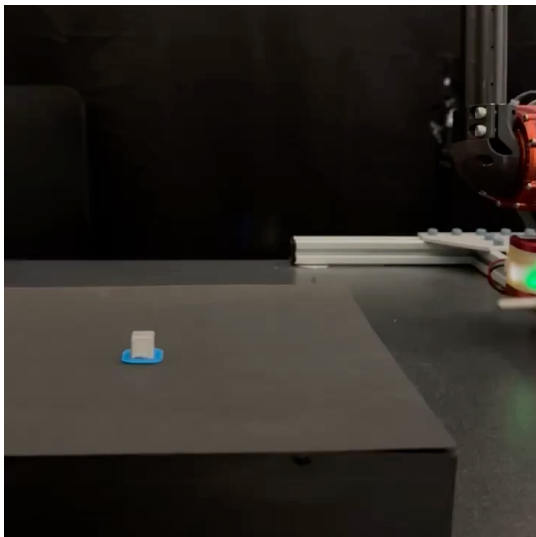


Without corrective labels

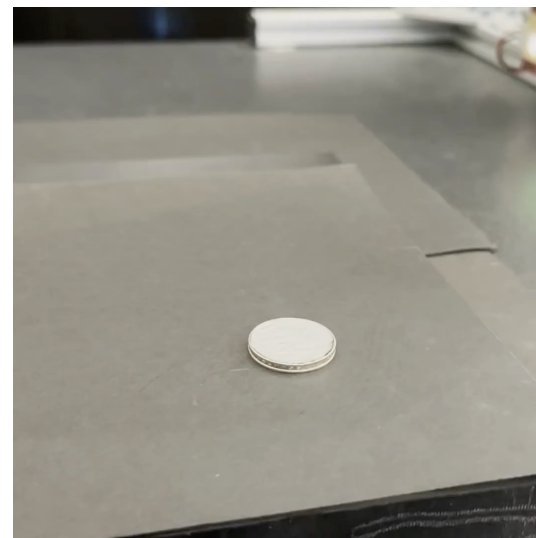
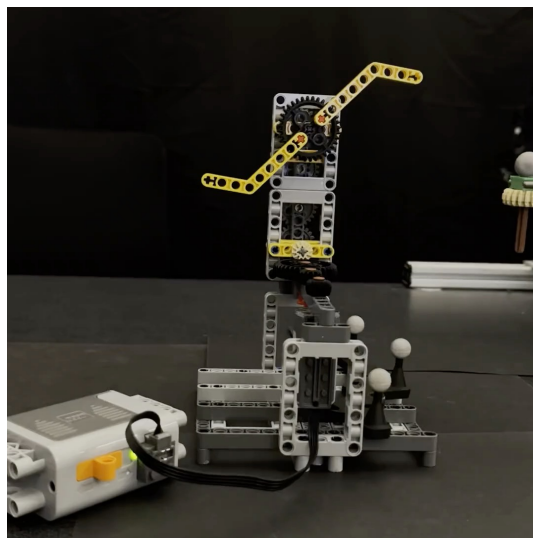
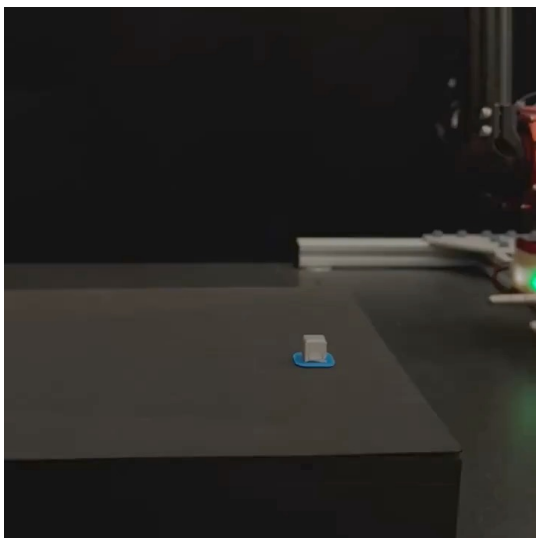


How well does generating corrective labels work?

With corrective labels



Without corrective labels



Lecture outline

Recap: MDP formalism + why should we care?



Imitation learning: preliminaries and behavior cloning



Multimodality and Underfitting in Imitation

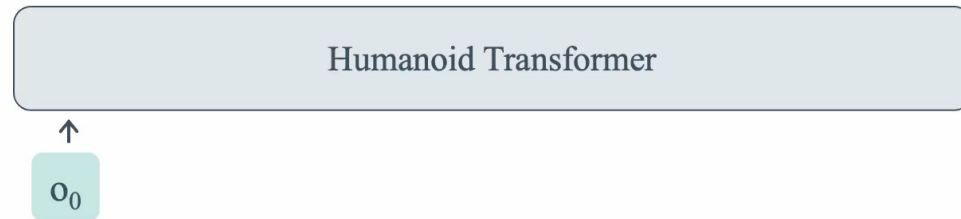


Compounding Error in Imitation

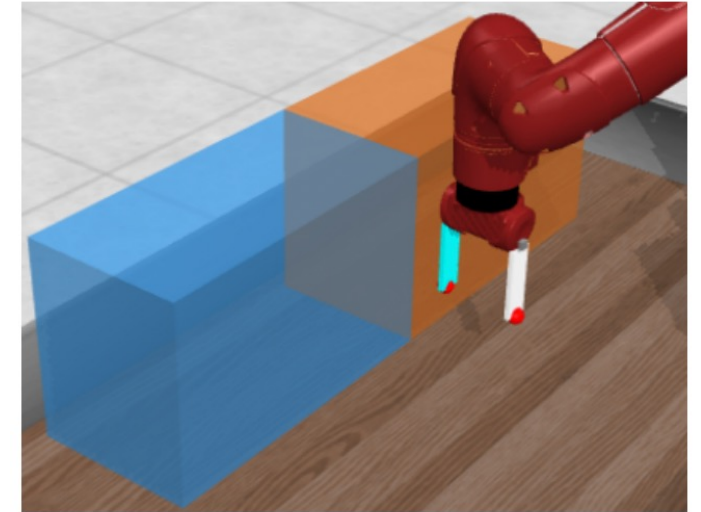
So does this solve all the issues in imitation?

Frontiers in Imitation Learning

Non-Markovian Demonstrators



Characterizing generalization

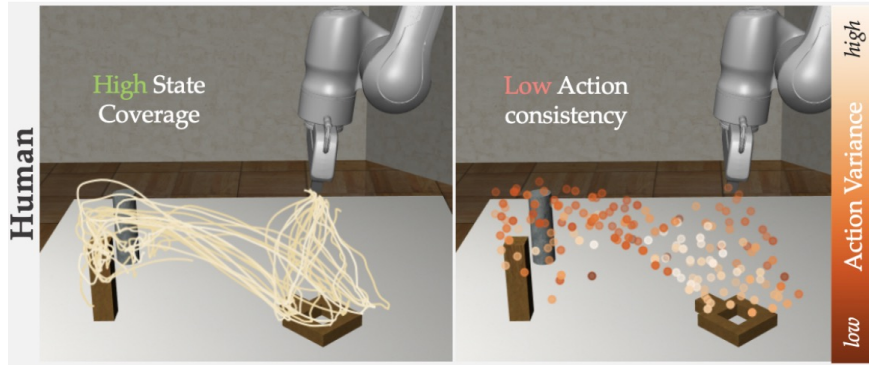


Action-Free Data

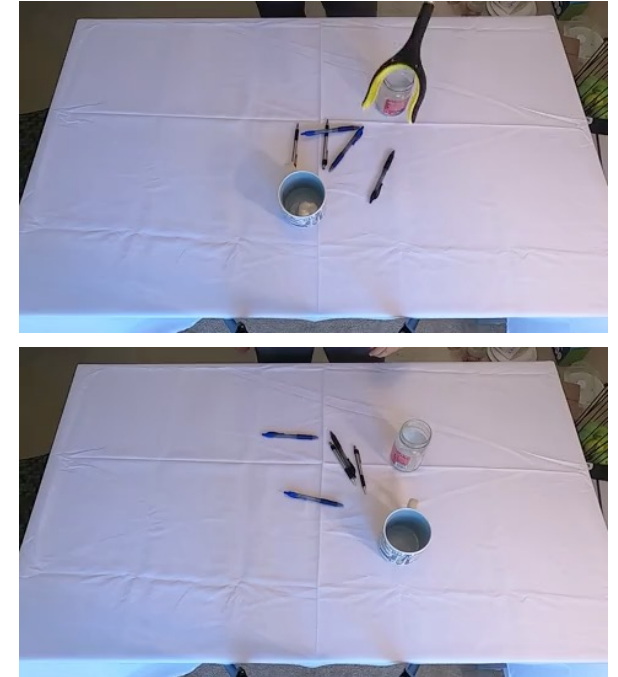


Frontiers in Imitation Learning

Data Curation and Quality



Embodiment Shift

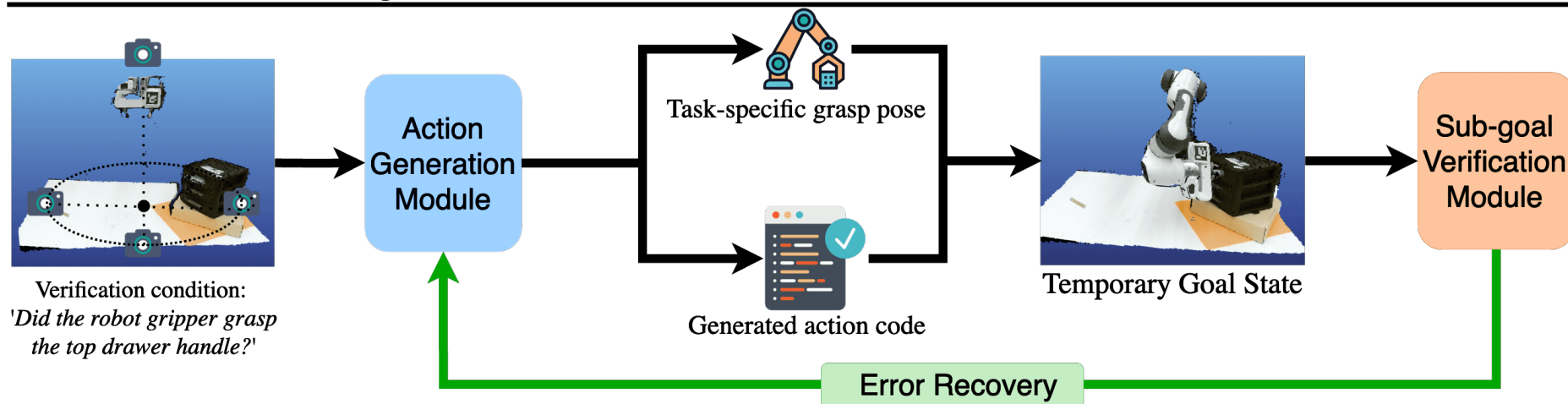
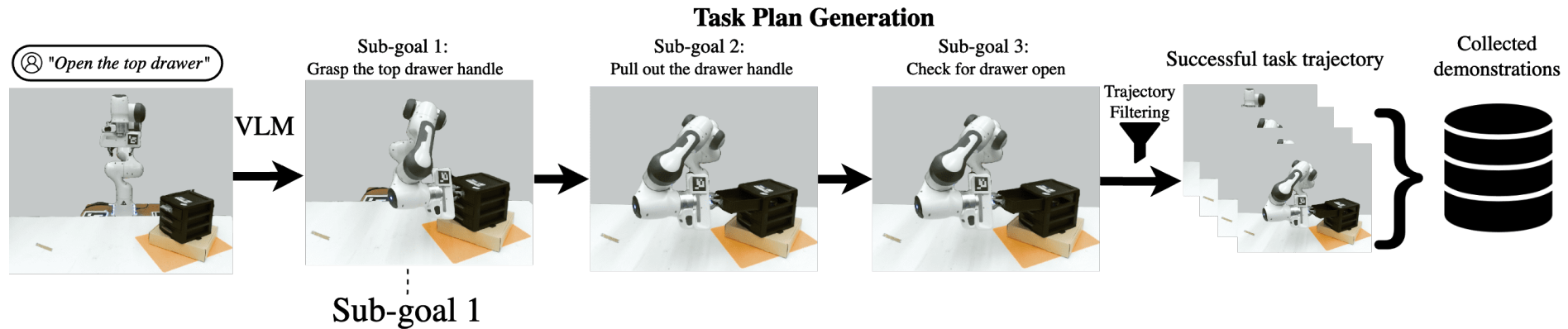


Teleoperation Interfaces



Frontiers in Imitation Learning

Learning how to retry and improve



Duan et al

Some cool imitation videos

1x and tesla humanoid robots



● 1X END-TO-END AUTONOMY
UPDATE, JAN 2024

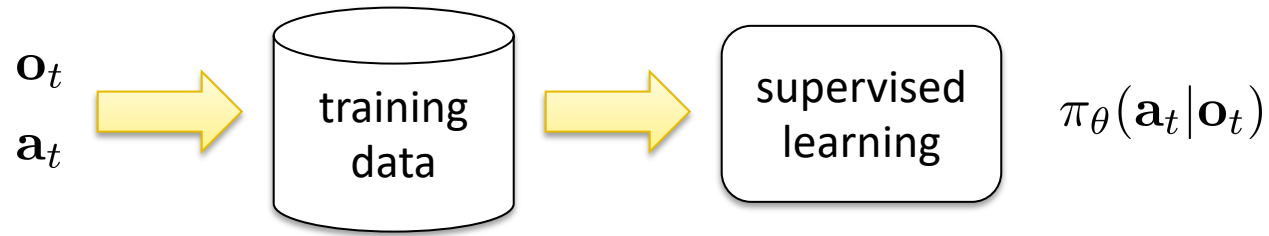
ALOHA and CherryBot Fine Manipulation



TRI Diffusion Policies

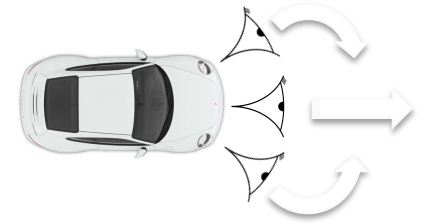


Perspectives on Imitation



■ Pros:

- Easy to use, no additional infra
- Can sometimes be unreasonably effective



■ Cons:

- Challenges of compounding error, multimodality
- Doesn't really generalize
- Very expensive in terms of data collection!

